DEVOPS ENGINEERING MASTERY

INFRASTRUCTURE AS CODE & TERRAFORM ENGINEERING

Infrastructure Automation, Terraform Architecture, State Management & Production Provisioning

VOLUME 08 • FIRST EDITION

```
nahid@technova:~$ terraform plan -out=tfplan
✓ Plan: 12 to add, 0 to change, 0 to destroy
nahid@technova:~$ terraform apply tfplan
✓ Apply complete! Resources: 12 added, 0 changed, 0 destroyed
nahid@technova:~$ terraform state list | wc -l
```

Terraform • HCL • AWS • Azure • GCP • IBM Cloud

First Edition • Published 9 November 2026 • Dhaka, Bangladesh

Companion Workbook • 24 Labs • 3 Projects • Capstone

Terraform Lab Workbook

DEVOPS ENGINEERING MASTERY — Volume 08 (Companion Workbook). First Edition, 9 November 2026. Terraform and HCL with AWS, Azure, GCP and IBM Cloud, built around the TechNova MERN example application. This workbook collects all 24 guided labs, 15 assignments, the 3 major projects and the capstone, so you can work through them beside your terminal without carrying the whole book. Do the labs in order; each lab states its prerequisites. Early labs use local providers so they cost nothing; cloud labs always warn about charges and end with a `terraform destroy` cleanup step. Never run a lab step on production infrastructure. The instructor-style solution guide for the capstone is at the end — attempt the capstone before reading it.

Lab Workbook • Volume 08

Lab Workbook — Infrastructure as Code & Terraform Engineering

এই ওয়ার্কবুক *DevOps Engineering Mastery — Volume* 08 বইয়ের হাতে-কলমে সঙ্গী। বইয়ে ধারণা পড়েছেন; এখানে নিজের হাতে করে শিখবেন। ল্যাবগুলো ক্রমানুসারে সাজানো — Lab 1 থেকে Lab 24, একটার উপর আরেকটা দাঁড়িয়ে আছে। প্রথম ১৬টি ল্যাব প্রায় সম্পূর্ণ লোকাল, তাই খরচ ছাড়াই বারবার অনুশীলন করা যায়। Lab 17–21 ও 24 ক্লাউডে কাজ করে; সেগুলোর আগে খরচের সতর্কতা পড়ুন, বাজেট অ্যালার্ট সেট করুন, শেষে destroy করুন।

১. ল্যাব পরিবেশ

উপাদান	সংস্করণ
অপারেটিং সিস্টেম	Ubuntu 24.04 LTS
Terraform	>= 1.9, < 2.0
প্রোভাইডার	aws ~> 5.0; azurerm ~> 4.0; google ~> 6.0; ibm ~> 1.0
প্রথম দিকের ল্যাব	local_file, null, random — লোকাল প্রোভাইডার

২. নিরাপত্তা নিয়ম

> **WARNING:** কখনো প্রোডাকশনে ল্যাবের কোড চালাবেন না। স্টেট ফাইল (`*.tfstate`), পাসওয়ার্ড, API কী বা ক্রেডেনশিয়াল Git–এ কমিট করবেন না। সিক্রেট পরিবেশ ভ্যারিয়েবল বা সিক্রেট ম্যানেজারে রাখুন। নমুনার সব মান — `CHANGE-ME-...`, `technova.example`, `203.0.113.x` — কাল্পনিক। কোনো অর্থপ্রদানকারী Terraform প্রশিক্ষণ বা কোর্স প্রয়োজন নেই।

প্রতিটি ল্যাব শেষে `terraform destroy` চালান; ক্লাউডে ছোট সাইজ ব্যবহার করুন; `0.0.0.0/0` দিয়ে SSH বা ডেটাবেস খুলবেন না।

৩. অধ্যয়নের ক্রম

Lab 1–9 (ভিত্তি) → Lab 10–11 (ব্যাকএন্ড ও লকিং) → Lab 12–15 (ভাষা ও মডিউল) → Lab 16 (ইমপোর্ট) → Lab 17–20 (চার ক্লাউড) → Lab 21–23 (IAM, নিরাপত্তা, CI/CD) → Lab 24 (সব একসঙ্গে) → Assignments → Projects → Capstone। সপ্তাহে ৩–৪টি ল্যাব করলে প্রায় ছয় সপ্তাহে শেষ হবে। প্রতিটি ল্যাবে একটি জিনিস নিজে বদলে `plan` দেখুন — সেটাই আসল শেখা।

Lab 1 • Volume 08

Lab 1 — Install Terraform and Inspect the CLI

এই ল্যাবে **Terraform ইনস্টল ও CLI পরিদর্শন** হাতে-কলমে শিখবেন (TechNova, `technova.example`, `203.0.113.x`)। প্রতিটি কমান্ডের আউটপুট নিজে পড়ুন।

১.১ উদ্দেশ্য, টুলস ও পূর্বশর্ত

ল্যাব শেষে পারবেন: Terraform ইনস্টল ও CLI পরিদর্শন–এর ধারণা ব্যাখ্যা করতে, ফাইল লিখে `init`, `plan` ও `apply` চালাতে, এবং আউটপুট ও স্টেট পড়ে ফল যাচাই করতে। প্রয়োজন: Ubuntu 24.04, Terraform >= 1.9, < 2.0, Git। আলাদা ডিরেক্টরি ব্যবহার করুন: `mkdir -p ~/labs/lab1 && cd ~/labs/lab1`। ক্লাউড ল্যাবে ক্রেডেনশিয়াল পরিবেশ ভ্যারিয়েবলে সেট করুন, কোডে নয়।

১.২ আর্কিটেকচার

```
You (Ubuntu 24.04, Terraform 1.9+) --init/plan/apply--> main.tf --> Provider --> terraform.tfst
```

১.৩ ধাপে ধাপে নির্দেশনা

ধাপ ১: ডিরেক্টরি তৈরি করুন। **ধাপ ২:** নিচের `main.tf` লিখুন। **ধাপ ৩:** `terraform fmt` ও `terraform validate` চালান। **ধাপ ৪:** `plan` পড়ে নিশ্চিত হয়ে `apply` দিন।

```
terraform {
  required_version = ">= 1.9, < 2.0"
}
```

```
terraform version
terraform -help
terraform fmt -check
```

> **খরচ:** সম্পূর্ণ লোকাল — কোনো ক্লাউড খরচ নেই।

১.৪ যাচাই, সমস্যা সমাধান ও পরিষ্কার

সফল হলে `Apply complete!` দেখবেন। যাচাই: `terraform state list; terraform output; terraform plan` — দ্বিতীয় plan–এ `No changes` আসা উচিত।

> **TROUBLESHOOTING:** `command not found` হলে Terraform আবার ইনস্টল করুন (Lab 1)। `Invalid reference` হলে নামের বানান দেখুন। `state lock` হলে অন্য প্রসেস দেখুন, তারপর `force-unlock`। ক্লাউডে `AccessDenied` হলে ক্রেডেনশিয়াল ও IAM পরীক্ষা করুন।

পরিষ্কার: `terraform destroy -auto-approve` চালিয়ে `Destroy complete!` দেখে থামুন; ক্লাউডে destroy না করলে বিল চলবে। `.gitignore`-এ `.terraform/`, `*.tfstate`, `*.tfvars` রাখুন — স্টেট বা সিক্রেট কখনো Git-এ নয়। **পর্যালোচনা:** ১) `plan` ও `apply`-এর পার্থক্য কী? ২) স্টেট ফাইল মুছে গেলে কী হবে? ৩) এই কোড প্রোডাকশনে ব্যবহার করতে কী বদলাতে হবে?

Lab 2 • Volume 08

Lab 2 — Create Your First Terraform Project

এই ল্যাবে **প্রথম Terraform প্রজেক্ট তৈরি** হাতে-কলমে শিখবেন (TechNova, `technova.example`, `203.0.113.x`)। প্রতিটি কমান্ডের আউটপুট নিজে পড়ুন।

২.১ উদ্দেশ্য, টুলস ও পূর্বশর্ত

ল্যাব শেষে পারবেন: প্রথম Terraform প্রজেক্ট তৈরি–এর ধারণা ব্যাখ্যা করতে, ফাইল লিখে `init`, `plan` ও `apply` চালাতে, এবং আউটপুট ও স্টেট পড়ে ফল যাচাই করতে। প্রয়োজন: Ubuntu 24.04, Terraform >= 1.9, < 2.0, Git। আলাদা ডিরেক্টরি ব্যবহার করুন: `mkdir -p ~/labs/lab2 && cd ~/labs/lab2`। ক্লাউড ল্যাবে ক্রেডেনশিয়াল পরিবেশ ভ্যারিয়েবলে সেট করুন, কোডে নয়।

২.২ আর্কিটেকচার

```
You (Ubuntu 24.04, Terraform 1.9+) --init/plan/apply--> main.tf --> Provider --> terraform.tfst
```

২.৩ ধাপে ধাপে নির্দেশনা

ধাপ ১: ডিরেক্টরি তৈরি করুন। **ধাপ ২:** নিচের `main.tf` লিখুন। **ধাপ ৩:** `terraform fmt` ও `terraform validate` চালান। **ধাপ ৪:** `plan` পড়ে নিশ্চিত হয়ে `apply` দিন।

```
terraform {
  required_version = ">= 1.9, < 2.0"
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

resource "local_file" "hello" {
  content  = "Hello, TechNova!\n"
  filename = "${path.module}/hello.txt"
}
```

```
terraform init
terraform plan
terraform apply -auto-approve
cat hello.txt
```

> **খরচ:** সম্পূর্ণ লোকাল — কোনো ক্লাউড খরচ নেই।

২.৪ যাচাই, সমস্যা সমাধান ও পরিষ্কার

সফল হলে `Apply complete!` দেখবেন। যাচাই: `terraform state list; terraform output; terraform plan` — দ্বিতীয় plan-এ `No changes` আসা উচিত।

> **TROUBLESHOOTING:** `command not found` হলে Terraform আবার ইনস্টল করুন (Lab 1)। `Invalid reference` হলে নামের বানান দেখুন। `state lock` হলে অন্য প্রসেস দেখুন, তারপর `force-unlock`। ক্লাউডে `AccessDenied` হলে ক্রেডেনশিয়াল ও IAM পরীক্ষা করুন।

পরিষ্কার: `terraform destroy -auto-approve` চালিয়ে `Destroy complete!` দেখে থামুন; ক্লাউডে destroy না করলে বিল চলবে। `.gitignore`-এ `.terraform/`, `*.tfstate`, `*.tfvars` রাখুন — স্টেট বা সিক্রেট কখনো Git-এ নয়। **পর্যালোচনা:** ১) `plan` ও `apply`-এর পার্থক্য কী? ২) স্টেট ফাইল মুছে গেলে কী হবে? ৩) এই কোড প্রোডাকশনে ব্যবহার করতে কী বদলাতে হবে?

Lab 3 • Volume 08

Lab 3 — Explore HCL Syntax

এই ল্যাবে **HCL সিনট্যাক্স অন্বেষণ** হাতে-কলমে শিখবেন (TechNova, `technova.example`, `203.0.113.x`)। প্রতিটি কমান্ডের আউটপুট নিজে পড়ুন।

৩.১ উদ্দেশ্য, টুলস ও পূর্বশর্ত

ল্যাব শেষে পারবেন: HCL সিনট্যাক্স অন্বেষণ–এর ধারণা ব্যাখ্যা করতে, ফাইল লিখে `init`, `plan` ও `apply` চালাতে, এবং আউটপুট ও স্টেট পড়ে ফল যাচাই করতে। প্রয়োজন: Ubuntu 24.04, Terraform >= 1.9, < 2.0, Git। আলাদা ডিরেক্টরি ব্যবহার করুন: `mkdir -p ~/labs/lab3 && cd ~/labs/lab3`। ক্লাউড ল্যাবে ক্রেডেনশিয়াল পরিবেশ ভ্যারিয়েবলে সেট করুন, কোডে নয়।

৩.২ আর্কিটেকচার

```
You (Ubuntu 24.04, Terraform 1.9+) --init/plan/apply--> main.tf --> Provider --> terraform.tfst
```

৩.৩ ধাপে ধাপে নির্দেশনা

ধাপ ১: ডিরেক্টরি তৈরি করুন। **ধাপ ২:** নিচের `main.tf` লিখুন। **ধাপ ৩:** `terraform fmt` ও `terraform validate` চালান। **ধাপ ৪:** `plan` পড়ে নিশ্চিত হয়ে `apply` দিন।

```
locals {
  app_name    = "technova"
  environment = "dev"
  tags = {
    Team    = "Platform"
    Project = "technova.example"
  }
}

variable "region" {
  type    = string
  default = "ap-south-1"
}

output "app" {
  value = local.app_name
}
```

```
terraform console
terraform fmt
terraform validate
```

> **খরচ:** সম্পূর্ণ লোকাল — কোনো ক্লাউড খরচ নেই।

৩.৪ যাচাই, সমস্যা সমাধান ও পরিষ্কার

সফল হলে `Apply complete!` দেখবেন। যাচাই: `terraform state list; terraform output; terraform plan` — দ্বিতীয় plan-এ `No changes` আসা উচিত।

> **TROUBLESHOOTING:** `command not found` হলে Terraform আবার ইনস্টল করুন (Lab 1)। `Invalid reference` হলে নামের বানান দেখুন। `state lock` হলে অন্য প্রসেস দেখুন, তারপর `force-unlock`। ক্লাউডে `AccessDenied` হলে ক্রেডেনশিয়াল ও IAM পরীক্ষা করুন।

পরিষ্কার: `terraform destroy -auto-approve` চালিয়ে `Destroy complete!` দেখে থামুন; ক্লাউডে destroy না করলে বিল চলবে। `.gitignore`-এ `.terraform/`, `*.tfstate`, `*.tfvars` রাখুন — স্টেট বা সিক্রেট কখনো Git-এ নয়। **পর্যালোচনা:** ১) `plan` ও `apply`-এর পার্থক্য কী? ২) স্টেট ফাইল মুছে গেলে কী হবে? ৩) এই কোড প্রোডাকশনে ব্যবহার করতে কী বদলাতে হবে?

Lab 4 • Volume 08

Lab 4 — Variables and Outputs

এই ল্যাবে **ভেরিয়েবল ও আউটপুট** হাতে-কলমে শিখবেন (TechNova, `technova.example`, `203.0.113.x`)। প্রতিটি কমান্ডের আউটপুট নিজে পড়ুন।

৪.১ উদ্দেশ্য, টুলস ও পূর্বশর্ত

ল্যাব শেষে পারবেন: ভেরিয়েবল ও আউটপুট–এর ধারণা ব্যাখ্যা করতে, ফাইল লিখে `init`, `plan` ও `apply` চালাতে, এবং আউটপুট ও স্টেট পড়ে ফল যাচাই করতে। প্রয়োজন: Ubuntu 24.04, Terraform >= 1.9, < 2.0, Git। আলাদা ডিরেক্টরি ব্যবহার করুন: `mkdir -p ~/labs/lab4 && cd ~/labs/lab4`। ক্লাউড ল্যাবে ক্রেডেনশিয়াল পরিবেশ ভ্যারিয়েবলে সেট করুন, কোডে নয়।

৪.২ আর্কিটেকচার

```
You (Ubuntu 24.04, Terraform 1.9+) --init/plan/apply--> main.tf --> Provider --> terraform.tfst
```

৪.৩ ধাপে ধাপে নির্দেশনা

ধাপ ১: ডিরেক্টরি তৈরি করুন। **ধাপ ২:** নিচের `main.tf` লিখুন। **ধাপ ৩:** `terraform fmt` ও `terraform validate` চালান। **ধাপ ৪:** `plan` পড়ে নিশ্চিত হয়ে `apply` দিন।

```
variable "instance_count" {
  type        = number
  description = "Number of instances to simulate"
  default     = 2
  validation {
    condition     = var.instance_count >= 1 && var.instance_count <= 5
    error_message = "Count must be between 1 and 5."
  }
}

variable "tags" {
  type = map(string)
  default = {
    env = "dev"
  }
}

output "count" {
  value = var.instance_count
}
```

```
terraform plan -var="instance_count=3"
terraform output
terraform output -json
```

> **খরচ:** সম্পূর্ণ লোকাল — কোনো ক্লাউড খরচ নেই।

৪.৪ যাচাই, সমস্যা সমাধান ও পরিষ্কার

সফল হলে `Apply complete!` দেখবেন। যাচাই: `terraform state list; terraform output; terraform plan` — দ্বিতীয় plan-এ `No changes` আসা উচিত।

> **TROUBLESHOOTING:** `command not found` হলে Terraform আবার ইনস্টল করুন (Lab 1)। `Invalid reference` হলে নামের বানান দেখুন। `state lock` হলে অন্য প্রসেস দেখুন, তারপর `force-unlock`। ক্লাউডে `AccessDenied` হলে ক্রেডেনশিয়াল ও IAM পরীক্ষা করুন।

পরিষ্কার: `terraform destroy -auto-approve` চালিয়ে `Destroy complete!` দেখে থামুন; ক্লাউডে destroy না করলে বিল চলবে। `.gitignore`-এ `.terraform/`, `*.tfstate`, `*.tfvars` রাখুন — স্টেট বা সিক্রেট কখনো Git-এ নয়। **পর্যালোচনা:** ১) `plan` ও `apply`-এর পার্থক্য কী? ২) স্টেট ফাইল মুছে গেলে কী হবে? ৩) এই কোড প্রোডাকশনে ব্যবহার করতে কী বদলাতে হবে?

Lab 5 • Volume 08

Lab 5 — Local File Provider

এই ল্যাবে **Local File প্রোভাইডার** হাতে-কলমে শিখবেন (TechNova, `technova.example`, `203.0.113.x`)। প্রতিটি কমান্ডের আউটপুট নিজে পড়ুন।

৫.১ উদ্দেশ্য, টুলস ও পূর্বশর্ত

ল্যাব শেষে পারবেন: Local File প্রোভাইডার-এর ধারণা ব্যাখ্যা করতে, ফাইল লিখে `init`, `plan` ও `apply` চালাতে, এবং আউটপুট ও স্টেট পড়ে ফল যাচাই করতে। প্রয়োজন: Ubuntu 24.04, Terraform >= 1.9, < 2.0, Git। আলাদা ডিরেক্টরি ব্যবহার করুন: `mkdir -p ~/labs/lab5 && cd ~/labs/lab5`। ক্লাউড ল্যাবে ক্রেডেনশিয়াল পরিবেশ ভ্যারিয়েবলে সেট করুন, কোডে নয়।

৫.২ আর্কিটেকচার

```
You (Ubuntu 24.04, Terraform 1.9+) --init/plan/apply--> main.tf --> Provider --> terraform.tfst
```

৫.৩ ধাপে ধাপে নির্দেশনা

ধাপ ১: ডিরেক্টরি তৈরি করুন। **ধাপ ২:** নিচের `main.tf` লিখুন। **ধাপ ৩:** `terraform fmt` ও `terraform validate` চালান। **ধাপ ৪:** `plan` পড়ে নিশ্চিত হয়ে `apply` দিন।

```
resource "local_file" "config" {
  content = templatefile("${path.module}/app.tftpl", {
    app_name = "technova"
    port     = 8080
  })
  filename = "${path.module}/generated/app.conf"
}

resource "local_file" "readme" {
  content  = "# TechNova\nGenerated by Terraform.\n"
  filename = "${path.module}/README-generated.md"
}
```

```
terraform init
terraform apply -auto-approve
ls -R generated/
cat generated/app.conf
```

> **খরচ:** সম্পূর্ণ লোকাল — কোনো ক্লাউড খরচ নেই।

৫.৪ যাচাই, সমস্যা সমাধান ও পরিষ্কার

সফল হলে `Apply complete!` দেখবেন। যাচাই: `terraform state list; terraform output; terraform plan` — দ্বিতীয় plan-এ `No changes` আসা উচিত।

> **TROUBLESHOOTING:** `command not found` হলে Terraform আবার ইনস্টল করুন (Lab 1)। `Invalid reference` হলে নামের বানান দেখুন। `state lock` হলে অন্য প্রসেস দেখুন, তারপর `force-unlock`। ক্লাউডে `AccessDenied` হলে ক্রেডেনশিয়াল ও IAM পরীক্ষা করুন।

পরিষ্কার: `terraform destroy -auto-approve` চালিয়ে `Destroy complete!` দেখে থামুন; ক্লাউডে destroy না করলে বিল চলবে। `.gitignore`-এ `.terraform/`, `*.tfstate`, `*.tfvars` রাখুন — স্টেট বা সিক্রেট কখনো Git-এ নয়। **পর্যালোচনা:** ১) `plan` ও `apply`-এর পার্থক্য কী? ২) স্টেট ফাইল মুছে গেলে কী হবে? ৩) এই কোড প্রোডাকশনে ব্যবহার করতে কী বদলাতে হবে?

Lab 6 • Volume 08

Lab 6 — Resource Dependency Graph

এই ল্যাবে **রিসোর্স নির্ভরতা গ্রাফ** হাতে-কলমে শিখবেন (TechNova, `technova.example`, `203.0.113.x`)। প্রতিটি কমান্ডের আউটপুট নিজে পড়ুন।

৬.১ উদ্দেশ্য, টুলস ও পূর্বশর্ত

ল্যাব শেষে পারবেন: রিসোর্স নির্ভরতা গ্রাফ–এর ধারণা ব্যাখ্যা করতে, ফাইল লিখে `init`, `plan` ও `apply` চালাতে, এবং আউটপুট ও স্টেট পড়ে ফল যাচাই করতে। প্রয়োজন: Ubuntu 24.04, Terraform >= 1.9, < 2.0, Git। আলাদা ডিরেক্টরি ব্যবহার করুন: `mkdir -p ~/labs/lab6 && cd ~/labs/lab6`। ক্লাউড ল্যাবে ক্রেডেনশিয়াল পরিবেশ ভ্যারিয়েবলে সেট করুন, কোডে নয়।

৬.২ আর্কিটেকচার

```
You (Ubuntu 24.04, Terraform 1.9+) --init/plan/apply--> main.tf --> Provider --> terraform.tfst
```

৬.৩ ধাপে ধাপে নির্দেশনা

ধাপ ১: ডিরেক্টরি তৈরি করুন। **ধাপ ২:** নিচের `main.tf` লিখুন। **ধাপ ৩:** `terraform fmt` ও `terraform validate` চালান। **ধাপ ৪:** `plan` পড়ে নিশ্চিত হয়ে `apply` দিন।

```
resource "local_file" "base" {
  content  = "base"
  filename = "${path.module}/base.txt"
}

resource "local_file" "child" {
  content    = local_file.base.content
  filename   = "${path.module}/child.txt"
  depends_on = [local_file.base]
}
```

```
terraform graph | dot -Tpng > graph.png
terraform plan
terraform apply -auto-approve
```

> **খরচ:** সম্পূর্ণ লোকাল — কোনো ক্লাউড খরচ নেই।

৬.৪ যাচাই, সমস্যা সমাধান ও পরিষ্কার

সফল হলে `Apply complete!` দেখবেন। যাচাই: `terraform state list; terraform output; terraform plan` — দ্বিতীয় plan-এ `No changes` আসা উচিত।

> **TROUBLESHOOTING:** `command not found` হলে Terraform আবার ইনস্টল করুন (Lab 1)। `Invalid reference` হলে নামের বানান দেখুন। `state lock` হলে অন্য প্রসেস দেখুন, তারপর `force-unlock`। ক্লাউডে `AccessDenied` হলে ক্রেডেনশিয়াল ও IAM পরীক্ষা করুন।

পরিষ্কার: `terraform destroy -auto-approve` চালিয়ে `Destroy complete!` দেখে থামুন; ক্লাউডে destroy না করলে বিল চলবে। `.gitignore`-এ `.terraform/`, `*.tfstate`, `*.tfvars` রাখুন — স্টেট বা সিক্রেট কখনো Git-এ নয়। **পর্যালোচনা:** ১) `plan` ও `apply`-এর পার্থক্য কী? ২) স্টেট ফাইল মুছে গেলে কী হবে? ৩) এই কোড প্রোডাকশনে ব্যবহার করতে কী বদলাতে হবে?

Lab 7 • Volume 08

Lab 7 — Data Sources

এই ল্যাবে **ডেটা সোর্স** হাতে-কলমে শিখবেন (TechNova, `technova.example`, `203.0.113.x`)। প্রতিটি কমান্ডের আউটপুট নিজে পড়ুন।

৭.১ উদ্দেশ্য, টুলস ও পূর্বশর্ত

ল্যাব শেষে পারবেন: ডেটা সোর্স–এর ধারণা ব্যাখ্যা করতে, ফাইল লিখে `init`, `plan` ও `apply` চালাতে, এবং আউটপুট ও স্টেট পড়ে ফল যাচাই করতে। প্রয়োজন: Ubuntu 24.04, Terraform >= 1.9, < 2.0, Git। আলাদা ডিরেক্টরি ব্যবহার করুন: `mkdir -p ~/labs/lab7 && cd ~/labs/lab7`। ক্লাউড ল্যাবে ক্রেডেনশিয়াল পরিবেশ ভ্যারিয়েবলে সেট করুন, কোডে নয়।

৭.২ আর্কিটেকচার

```
You (Ubuntu 24.04, Terraform 1.9+) --init/plan/apply--> main.tf --> Provider --> terraform.tfst
```

৭.৩ ধাপে ধাপে নির্দেশনা

ধাপ ১: ডিরেক্টরি তৈরি করুন। **ধাপ ২:** নিচের `main.tf` লিখুন। **ধাপ ৩:** `terraform fmt` ও `terraform validate` চালান। **ধাপ ৪:** `plan` পড়ে নিশ্চিত হয়ে `apply` দিন।

```
data "local_file" "existing" {
  filename = "${path.module}/input.txt"
}

locals {
  lines = split("\n", data.local_file.existing.content)
}

output "first_line" {
  value = local.lines[0]
}
```

```
echo "hello-technova" > input.txt
terraform init
terraform plan
terraform apply -auto-approve
```

> **খরচ:** সম্পূর্ণ লোকাল — কোনো ক্লাউড খরচ নেই।

৭.৪ যাচাই, সমস্যা সমাধান ও পরিষ্কার

সফল হলে `Apply complete!` দেখবেন। যাচাই: `terraform state list; terraform output; terraform plan` — দ্বিতীয় plan-এ `No changes` আসা উচিত।

> **TROUBLESHOOTING:** `command not found` হলে Terraform আবার ইনস্টল করুন (Lab 1)। `Invalid reference` হলে নামের বানান দেখুন। `state lock` হলে অন্য প্রসেস দেখুন, তারপর `force-unlock`। ক্লাউডে `AccessDenied` হলে ক্রেডেনশিয়াল ও IAM পরীক্ষা করুন।

পরিষ্কার: `terraform destroy -auto-approve` চালিয়ে `Destroy complete!` দেখে থামুন; ক্লাউডে destroy না করলে বিল চলবে। `.gitignore`-এ `.terraform/`, `*.tfstate`, `*.tfvars` রাখুন — স্টেট বা সিক্রেট কখনো Git-এ নয়। **পর্যালোচনা:** ১) `plan` ও `apply`-এর পার্থক্য কী? ২) স্টেট ফাইল মুছে গেলে কী হবে? ৩) এই কোড প্রোডাকশনে ব্যবহার করতে কী বদলাতে হবে?

Lab 8 • Volume 08

Lab 8 — Plan and Apply

এই ল্যাবে **Plan ও Apply** হাতে-কলমে শিখবেন (TechNova, `technova.example`, `203.0.113.x`)। প্রতিটি কমান্ডের আউটপুট নিজে পড়ুন।

৮.১ উদ্দেশ্য, টুলস ও পূর্বশর্ত

ল্যাব শেষে পারবেন: Plan ও Apply-এর ধারণা ব্যাখ্যা করতে, ফাইল লিখে `init`, `plan` ও `apply` চালাতে, এবং আউটপুট ও স্টেট পড়ে ফল যাচাই করতে। প্রয়োজন: Ubuntu 24.04, Terraform >= 1.9, < 2.0, Git। আলাদা ডিরেক্টরি ব্যবহার করুন: `mkdir -p ~/labs/lab8 && cd ~/labs/lab8`। ক্লাউড ল্যাবে ক্রেডেনশিয়াল পরিবেশ ভ্যারিয়েবলে সেট করুন, কোডে নয়।

৮.২ আর্কিটেকচার

```
You (Ubuntu 24.04, Terraform 1.9+) --init/plan/apply--> main.tf --> Provider --> terraform.tfst
```

৮.৩ ধাপে ধাপে নির্দেশনা

ধাপ ১: ডিরেক্টরি তৈরি করুন। **ধাপ ২:** নিচের `main.tf` লিখুন। **ধাপ ৩:** `terraform fmt` ও `terraform validate` চালান। **ধাপ ৪:** `plan` পড়ে নিশ্চিত হয়ে `apply` দিন।

```
resource "random_pet" "name" {
  length = 2
}

resource "local_file" "pet" {
  content  = random_pet.name.id
  filename = "${path.module}/pet.txt"
}
```

```
terraform plan -out=tfplan
terraform show tfplan
terraform apply tfplan
terraform plan -detailed-exitcode
```

> **খরচ:** সম্পূর্ণ লোকাল — কোনো ক্লাউড খরচ নেই।

৮.৪ যাচাই, সমস্যা সমাধান ও পরিষ্কার

সফল হলে `Apply complete!` দেখবেন। যাচাই: `terraform state list; terraform output; terraform plan` — দ্বিতীয় plan-এ `No changes` আসা উচিত।

> **TROUBLESHOOTING:** `command not found` হলে Terraform আবার ইনস্টল করুন (Lab 1)। `Invalid reference` হলে নামের বানান দেখুন। `state lock` হলে অন্য প্রসেস দেখুন, তারপর `force-unlock`। ক্লাউডে `AccessDenied` হলে ক্রেডেনশিয়াল ও IAM পরীক্ষা করুন।

পরিষ্কার: `terraform destroy -auto-approve` চালিয়ে `Destroy complete!` দেখে থামুন; ক্লাউডে destroy না করলে বিল চলবে। `.gitignore`-এ `.terraform/`, `*.tfstate`, `*.tfvars` রাখুন — স্টেট বা সিক্রেট কখনো Git-এ নয়। **পর্যালোচনা:** ১) `plan` ও `apply`-এর পার্থক্য কী? ২) স্টেট ফাইল মুছে গেলে কী হবে? ৩) এই কোড প্রোডাকশনে ব্যবহার করতে কী বদলাতে হবে?

Lab 9 • Volume 08

Lab 9 — State File Inspection

এই ল্যাবে **স্টেট ফাইল পরিদর্শন** হাতে-কলমে শিখবেন (TechNova, `technova.example`, `203.0.113.x`)। প্রতিটি কমান্ডের আউটপুট নিজে পড়ুন।

৯.১ উদ্দেশ্য, টুলস ও পূর্বশর্ত

ল্যাব শেষে পারবেন: স্টেট ফাইল পরিদর্শন-এর ধারণা ব্যাখ্যা করতে, ফাইল লিখে `init`, `plan` ও `apply` চালাতে, এবং আউটপুট ও স্টেট পড়ে ফল যাচাই করতে। প্রয়োজন: Ubuntu 24.04, Terraform >= 1.9, < 2.0, Git। আলাদা ডিরেক্টরি ব্যবহার করুন: `mkdir -p ~/labs/lab9 && cd ~/labs/lab9`। ক্লাউড ল্যাবে ক্রেডেনশিয়াল পরিবেশ ভ্যারিয়েবলে সেট করুন, কোডে নয়।

৯.২ আর্কিটেকচার

```
You (Ubuntu 24.04, Terraform 1.9+) --init/plan/apply--> main.tf --> Provider --> terraform.tfst
```

৯.৩ ধাপে ধাপে নির্দেশনা

ধাপ ১: ডিরেক্টরি তৈরি করুন। **ধাপ ২:** নিচের `main.tf` লিখুন। **ধাপ ৩:** `terraform fmt` ও `terraform validate` চালান। **ধাপ ৪:** `plan` পড়ে নিশ্চিত হয়ে `apply` দিন।

```
resource "local_file" "a" {
  content  = "state-demo"
  filename = "${path.module}/a.txt"
}
```

```
terraform state list
terraform state show local_file.a
terraform state pull | head -60
terraform refresh
```

> **খরচ:** সম্পূর্ণ লোকাল — কোনো ক্লাউড খরচ নেই।

৯.৪ যাচাই, সমস্যা সমাধান ও পরিষ্কার

সফল হলে `Apply complete!` দেখবেন। যাচাই: `terraform state list; terraform output; terraform plan` — দ্বিতীয় plan-এ `No changes` আসা উচিত।

> **TROUBLESHOOTING:** `command not found` হলে Terraform আবার ইনস্টল করুন (Lab 1)। `Invalid reference` হলে নামের বানান দেখুন। `state lock` হলে অন্য প্রসেস দেখুন, তারপর `force-unlock`। ক্লাউডে `AccessDenied` হলে ক্রেডেনশিয়াল ও IAM পরীক্ষা করুন।

পরিষ্কার: `terraform destroy -auto-approve` চালিয়ে `Destroy complete!` দেখে থামুন; ক্লাউডে destroy না করলে বিল চলবে। `.gitignore`-এ `.terraform/`, `*.tfstate`, `*.tfvars` রাখুন — স্টেট বা সিক্রেট কখনো Git-এ নয়। **পর্যালোচনা:** ১) `plan` ও `apply`-এর পার্থক্য কী? ২) স্টেট ফাইল মুছে গেলে কী হবে? ৩) এই কোড প্রোডাকশনে ব্যবহার করতে কী বদলাতে হবে?

Lab 10 • Volume 08

Lab 10 — Remote Backend

এই ল্যাবে **রিমোট ব্যাকএন্ড** হাতে-কলমে শিখবেন (TechNova, `technova.example`, `203.0.113.x`)। প্রতিটি কমান্ডের আউটপুট নিজে পড়ুন।

১০.১ উদ্দেশ্য, টুলস ও পূর্বশর্ত

ল্যাব শেষে পারবেন: রিমোট ব্যাকএন্ড–এর ধারণা ব্যাখ্যা করতে, ফাইল লিখে `init`, `plan` ও `apply` চালাতে, এবং আউটপুট ও স্টেট পড়ে ফল যাচাই করতে। প্রয়োজন: Ubuntu 24.04, Terraform >= 1.9, < 2.0, Git। আলাদা ডিরেক্টরি ব্যবহার করুন: `mkdir -p ~/labs/lab10 && cd ~/labs/lab10`। ক্লাউড ল্যাবে ক্রেডেনশিয়াল পরিবেশ ভ্যারিয়েবলে সেট করুন, কোডে নয়।

১০.২ আর্কিটেকচার

```
You (Ubuntu 24.04, Terraform 1.9+) --init/plan/apply--> main.tf --> Provider --> terraform.tfst
```

১০.৩ ধাপে ধাপে নির্দেশনা

ধাপ ১: ডিরেক্টরি তৈরি করুন। **ধাপ ২:** নিচের `main.tf` লিখুন। **ধাপ ৩:** `terraform fmt` ও `terraform validate` চালান। **ধাপ ৪:** `plan` পড়ে নিশ্চিত হয়ে `apply` দিন।

```
terraform {
  backend "local" {
    path = "terraform.tfstate"
  }
}
# Production example (do not apply without credentials):
# terraform {
#   backend "s3" {
#     bucket         = "CHANGE-ME-technova-tfstate"
#     key            = "dev/terraform.tfstate"
#     region         = "ap-south-1"
#     dynamodb_table = "CHANGE-ME-technova-locks"
#     encrypt        = true
#   }
# }
```

```
terraform init
terraform init -migrate-state
terraform state list
```

> **খরচ:** সম্পূর্ণ লোকাল — কোনো ক্লাউড খরচ নেই।

১০.৪ যাচাই, সমস্যা সমাধান ও পরিষ্কার

সফল হলে `Apply complete!` দেখবেন। যাচাই: `terraform state list; terraform output; terraform plan` — দ্বিতীয় plan-এ `No changes` আসা উচিত।

> **TROUBLESHOOTING:** `command not found` হলে Terraform আবার ইনস্টল করুন (Lab 1)। `Invalid reference` হলে নামের বানান দেখুন। `state lock` হলে অন্য প্রসেস দেখুন, তারপর `force-unlock`। ক্লাউডে `AccessDenied` হলে ক্রেডেনশিয়াল ও IAM পরীক্ষা করুন।

পরিষ্কার: `terraform destroy -auto-approve` চালিয়ে `Destroy complete!` দেখে থামুন; ক্লাউডে destroy না করলে বিল চলবে। `.gitignore`-এ `.terraform/`, `*.tfstate`, `*.tfvars` রাখুন — স্টেট বা সিক্রেট কখনো Git-এ নয়। **পর্যালোচনা:** ১) `plan` ও `apply`-এর পার্থক্য কী? ২) স্টেট ফাইল মুছে গেলে কী হবে? ৩) এই কোড প্রোডাকশনে ব্যবহার করতে কী বদলাতে হবে?

Lab 11 • Volume 08

Lab 11 — State Locking

এই ল্যাবে **স্টেট লকিং** হাতে-কলমে শিখবেন (TechNova, `technova.example`, `203.0.113.x`)। প্রতিটি কমান্ডের আউটপুট নিজে পড়ুন।

১১.১ উদ্দেশ্য, টুলস ও পূর্বশর্ত

ল্যাব শেষে পারবেন: স্টেট লকিং-এর ধারণা ব্যাখ্যা করতে, ফাইল লিখে `init`, `plan` ও `apply` চালাতে, এবং আউটপুট ও স্টেট পড়ে ফল যাচাই করতে। প্রয়োজন: Ubuntu 24.04, Terraform >= 1.9, < 2.0, Git। আলাদা ডিরেক্টরি ব্যবহার করুন: `mkdir -p ~/labs/lab11 && cd ~/labs/lab11`। ক্লাউড ল্যাবে ক্রেডেনশিয়াল পরিবেশ ভ্যারিয়েবলে সেট করুন, কোডে নয়।

১১.২ আর্কিটেকচার

```
You (Ubuntu 24.04, Terraform 1.9+) --init/plan/apply--> main.tf --> Provider --> terraform.tfst
```

১১.৩ ধাপে ধাপে নির্দেশনা

ধাপ ১: ডিরেক্টরি তৈরি করুন। **ধাপ ২:** নিচের `main.tf` লিখুন। **ধাপ ৩:** `terraform fmt` ও `terraform validate` চালান। **ধাপ ৪:** `plan` পড়ে নিশ্চিত হয়ে `apply` দিন।

```
terraform {
  required_version = ">= 1.9, < 2.0"
  backend "local" {
    path = "terraform.tfstate"
  }
}
```

```
terraform apply -auto-approve
terraform force-unlock CHANGE-ME-LOCK-ID
terraform plan -lock-timeout=5m
```

> **খরচ:** সম্পূর্ণ লোকাল — কোনো ক্লাউড খরচ নেই।

১১.৪ যাচাই, সমস্যা সমাধান ও পরিষ্কার

সফল হলে `Apply complete!` দেখবেন। যাচাই: `terraform state list; terraform output; terraform plan` — দ্বিতীয় plan-এ `No changes` আসা উচিত।

> **TROUBLESHOOTING:** `command not found` হলে Terraform আবার ইনস্টল করুন (Lab 1)। `Invalid reference` হলে নামের বানান দেখুন। `state lock` হলে অন্য প্রসেস দেখুন, তারপর `force-unlock`। ক্লাউডে `AccessDenied` হলে ক্রেডেনশিয়াল ও IAM পরীক্ষা করুন।

পরিষ্কার: `terraform destroy -auto-approve` চালিয়ে `Destroy complete!` দেখে থামুন; ক্লাউডে destroy না করলে বিল চলবে। `.gitignore`-এ `.terraform/`, `*.tfstate`, `*.tfvars` রাখুন — স্টেট বা সিক্রেট কখনো Git-এ নয়। **পর্যালোচনা:** ১) `plan` ও `apply`-এর পার্থক্য কী? ২) স্টেট ফাইল মুছে গেলে কী হবে? ৩) এই কোড প্রোডাকশনে ব্যবহার করতে কী বদলাতে হবে?

Lab 12 • Volume 08

Lab 12 — count and for_each

এই ল্যাবে **count ও for_each** হাতে-কলমে শিখবেন (TechNova, `technova.example`, `203.0.113.x`)। প্রতিটি কমান্ডের আউটপুট নিজে পড়ুন।

১২.১ উদ্দেশ্য, টুলস ও পূর্বশর্ত

ল্যাব শেষে পারবেন: count ও for_each-এর ধারণা ব্যাখ্যা করতে, ফাইল লিখে `init`, `plan` ও `apply` চালাতে, এবং আউটপুট ও স্টেট পড়ে ফল যাচাই করতে। প্রয়োজন: Ubuntu 24.04, Terraform >= 1.9, < 2.0, Git। আলাদা ডিরেক্টরি ব্যবহার করুন: `mkdir -p ~/labs/lab12 && cd ~/labs/lab12`। ক্লাউড ল্যাবে ক্রেডেনশিয়াল পরিবেশ ভ্যারিয়েবলে সেট করুন, কোডে নয়।

১২.২ আর্কিটেকচার

```
You (Ubuntu 24.04, Terraform 1.9+) --init/plan/apply--> main.tf --> Provider --> terraform.tfst
```

১২.৩ ধাপে ধাপে নির্দেশনা

ধাপ ১: ডিরেক্টরি তৈরি করুন। **ধাপ ২:** নিচের `main.tf` লিখুন। **ধাপ ৩:** `terraform fmt` ও `terraform validate` চালান। **ধাপ ৪:** `plan` পড়ে নিশ্চিত হয়ে `apply` দিন।

```
variable "users" {
  type    = list(string)
  default = ["arifa", "rahim", "nusrat"]
}

resource "local_file" "by_count" {
  count    = 2
  content  = "server-${count.index}"
  filename = "${path.module}/server-${count.index}.txt"
}

resource "local_file" "by_each" {
  for_each = toset(var.users)
  content  = "user: ${each.key}"
  filename = "${path.module}/user-${each.key}.txt"
}
```

```
terraform plan
terraform apply -auto-approve
terraform state list
```

> **খরচ:** সম্পূর্ণ লোকাল — কোনো ক্লাউড খরচ নেই।

১২.৪ যাচাই, সমস্যা সমাধান ও পরিষ্কার

সফল হলে `Apply complete!` দেখবেন। যাচাই: `terraform state list; terraform output; terraform plan` — দ্বিতীয় plan-এ `No changes` আসা উচিত।

> **TROUBLESHOOTING:** `command not found` হলে Terraform আবার ইনস্টল করুন (Lab 1)। `Invalid reference` হলে নামের বানান দেখুন। `state lock` হলে অন্য প্রসেস দেখুন, তারপর `force-unlock`। ক্লাউডে `AccessDenied` হলে ক্রেডেনশিয়াল ও IAM পরীক্ষা করুন।

পরিষ্কার: `terraform destroy -auto-approve` চালিয়ে `Destroy complete!` দেখে থামুন; ক্লাউডে destroy না করলে বিল চলবে। `.gitignore`-এ `.terraform/`, `*.tfstate`, `*.tfvars` রাখুন — স্টেট বা সিক্রেট কখনো Git-এ নয়। **পর্যালোচনা:** ১) `plan` ও `apply`-এর পার্থক্য কী? ২) স্টেট ফাইল মুছে গেলে কী হবে? ৩) এই কোড প্রোডাকশনে ব্যবহার করতে কী বদলাতে হবে?

Lab 13 • Volume 08

Lab 13 — Dynamic Blocks

এই ল্যাবে **ডাইনামিক ব্লক** হাতে-কলমে শিখবেন (TechNova, `technova.example`, `203.0.113.x`)। প্রতিটি কমান্ডের আউটপুট নিজে পড়ুন।

১৩.১ উদ্দেশ্য, টুলস ও পূর্বশর্ত

ল্যাব শেষে পারবেন: ডাইনামিক ব্লক–এর ধারণা ব্যাখ্যা করতে, ফাইল লিখে `init`, `plan` ও `apply` চালাতে, এবং আউটপুট ও স্টেট পড়ে ফল যাচাই করতে। প্রয়োজন: Ubuntu 24.04, Terraform >= 1.9, < 2.0, Git। আলাদা ডিরেক্টরি ব্যবহার করুন: `mkdir -p ~/labs/lab13 && cd ~/labs/lab13`। ক্লাউড ল্যাবে ক্রেডেনশিয়াল পরিবেশ ভ্যারিয়েবলে সেট করুন, কোডে নয়।

১৩.২ আর্কিটেকচার

```
You (Ubuntu 24.04, Terraform 1.9+) --init/plan/apply--> main.tf --> Provider --> terraform.tfst
```

১৩.৩ ধাপে ধাপে নির্দেশনা

ধাপ ১: ডিরেক্টরি তৈরি করুন। **ধাপ ২:** নিচের `main.tf` লিখুন। **ধাপ ৩:** `terraform fmt` ও `terraform validate` চালান। **ধাপ ৪:** `plan` পড়ে নিশ্চিত হয়ে `apply` দিন।

```
variable "ports" {
  type    = list(number)
  default = [80, 443, 8080]
}

locals {
  rules = [for p in var.ports : {
    port = p
    desc = "Allow port ${p}"
  }]
}

output "rules" {
  value = local.rules
}
```

```
terraform plan
terraform apply -auto-approve
terraform output -json
```

> **খরচ:** সম্পূর্ণ লোকাল — কোনো ক্লাউড খরচ নেই।

১৩.৪ যাচাই, সমস্যা সমাধান ও পরিষ্কার

সফল হলে `Apply complete!` দেখবেন। যাচাই: `terraform state list; terraform output; terraform plan` — দ্বিতীয় plan-এ `No changes` আসা উচিত।

> **TROUBLESHOOTING:** `command not found` হলে Terraform আবার ইনস্টল করুন (Lab 1)। `Invalid reference` হলে নামের বানান দেখুন। `state lock` হলে অন্য প্রসেস দেখুন, তারপর `force-unlock`। ক্লাউডে `AccessDenied` হলে ক্রেডেনশিয়াল ও IAM পরীক্ষা করুন।

পরিষ্কার: `terraform destroy -auto-approve` চালিয়ে `Destroy complete!` দেখে থামুন; ক্লাউডে destroy না করলে বিল চলবে। `.gitignore`-এ `.terraform/`, `*.tfstate`, `*.tfvars` রাখুন — স্টেট বা সিক্রেট কখনো Git-এ নয়। **পর্যালোচনা:** ১) `plan` ও `apply`-এর পার্থক্য কী? ২) স্টেট ফাইল মুছে গেলে কী হবে? ৩) এই কোড প্রোডাকশনে ব্যবহার করতে কী বদলাতে হবে?

Lab 14 • Volume 08

Lab 14 — Reusable Modules

এই ল্যাবে **পুনর্ব্যবহারযোগ্য মডিউল** হাতে-কলমে শিখবেন (TechNova, `technova.example`, `203.0.113.x`)। প্রতিটি কমান্ডের আউটপুট নিজে পড়ুন।

১৪.১ উদ্দেশ্য, টুলস ও পূর্বশর্ত

ল্যাব শেষে পারবেন: পুনর্ব্যবহারযোগ্য মডিউল–এর ধারণা ব্যাখ্যা করতে, ফাইল লিখে `init`, `plan` ও `apply` চালাতে, এবং আউটপুট ও স্টেট পড়ে ফল যাচাই করতে। প্রয়োজন: Ubuntu 24.04, Terraform >= 1.9, < 2.0, Git। আলাদা ডিরেক্টরি ব্যবহার করুন: `mkdir -p ~/labs/lab14 && cd ~/labs/lab14`। ক্লাউড ল্যাবে ক্রেডেনশিয়াল পরিবেশ ভ্যারিয়েবলে সেট করুন, কোডে নয়।

১৪.২ আর্কিটেকচার

```
You (Ubuntu 24.04, Terraform 1.9+) --init/plan/apply--> main.tf --> Provider --> terraform.tfst
```

১৪.৩ ধাপে ধাপে নির্দেশনা

ধাপ ১: ডিরেক্টরি তৈরি করুন। **ধাপ ২:** নিচের `main.tf` লিখুন। **ধাপ ৩:** `terraform fmt` ও `terraform validate` চালান। **ধাপ ৪:** `plan` পড়ে নিশ্চিত হয়ে `apply` দিন।

```
# modules/greeting/main.tf
variable "name" {
  type = string
}
variable "message" {
  type    = string
  default = "Welcome"
}
resource "local_file" "greeting" {
  content  = "${var.message}, ${var.name}!"
  filename = "${path.module}/../../greeting-${var.name}.txt"
}
output "file" {
  value = local_file.greeting.filename
}
```

```
terraform init
terraform plan
terraform apply -auto-approve
```

> **খরচ:** সম্পূর্ণ লোকাল — কোনো ক্লাউড খরচ নেই।

১৪.৪ যাচাই, সমস্যা সমাধান ও পরিষ্কার

সফল হলে `Apply complete!` দেখবেন। যাচাই: `terraform state list; terraform output; terraform plan` — দ্বিতীয় plan-এ `No changes` আসা উচিত।

> **TROUBLESHOOTING:** `command not found` হলে Terraform আবার ইনস্টল করুন (Lab 1)। `Invalid reference` হলে নামের বানান দেখুন। `state lock` হলে অন্য প্রসেস দেখুন, তারপর `force-unlock`। ক্লাউডে `AccessDenied` হলে ক্রেডেনশিয়াল ও IAM পরীক্ষা করুন।

পরিষ্কার: `terraform destroy -auto-approve` চালিয়ে `Destroy complete!` দেখে থামুন; ক্লাউডে destroy না করলে বিল চলবে। `.gitignore`-এ `.terraform/`, `*.tfstate`, `*.tfvars` রাখুন — স্টেট বা সিক্রেট কখনো Git-এ নয়। **পর্যালোচনা:** ১) `plan` ও `apply`-এর পার্থক্য কী? ২) স্টেট ফাইল মুছে গেলে কী হবে? ৩) এই কোড প্রোডাকশনে ব্যবহার করতে কী বদলাতে হবে?

Lab 15 • Volume 08

Lab 15 — Module Composition

এই ল্যাবে **মডিউল কম্পোজিশন** হাতে-কলমে শিখবেন (TechNova, `technova.example`, `203.0.113.x`)। প্রতিটি কমান্ডের আউটপুট নিজে পড়ুন।

১৫.১ উদ্দেশ্য, টুলস ও পূর্বশর্ত

ল্যাব শেষে পারবেন: মডিউল কম্পোজিশন–এর ধারণা ব্যাখ্যা করতে, ফাইল লিখে `init`, `plan` ও `apply` চালাতে, এবং আউটপুট ও স্টেট পড়ে ফল যাচাই করতে। প্রয়োজন: Ubuntu 24.04, Terraform >= 1.9, < 2.0, Git। আলাদা ডিরেক্টরি ব্যবহার করুন: `mkdir -p ~/labs/lab15 && cd ~/labs/lab15`। ক্লাউড ল্যাবে ক্রেডেনশিয়াল পরিবেশ ভ্যারিয়েবলে সেট করুন, কোডে নয়।

১৫.২ আর্কিটেকচার

```
You (Ubuntu 24.04, Terraform 1.9+) --init/plan/apply--> main.tf --> Provider --> terraform.tfst
```

১৫.৩ ধাপে ধাপে নির্দেশনা

ধাপ ১: ডিরেক্টরি তৈরি করুন। **ধাপ ২:** নিচের `main.tf` লিখুন। **ধাপ ৩:** `terraform fmt` ও `terraform validate` চালান। **ধাপ ৪:** `plan` পড়ে নিশ্চিত হয়ে `apply` দিন।

```
module "app" {
  source = "./modules/app"
  name   = "technova"
  env    = "dev"
}

module "db" {
  source   = "./modules/db"
  app_name = module.app.name
}
```

```
terraform init
terraform plan
terraform apply -auto-approve
terraform output
```

> **খরচ:** সম্পূর্ণ লোকাল — কোনো ক্লাউড খরচ নেই।

১৫.৪ যাচাই, সমস্যা সমাধান ও পরিষ্কার

সফল হলে `Apply complete!` দেখবেন। যাচাই: `terraform state list; terraform output; terraform plan` — দ্বিতীয় plan-এ `No changes` আসা উচিত।

> **TROUBLESHOOTING:** `command not found` হলে Terraform আবার ইনস্টল করুন (Lab 1)। `Invalid reference` হলে নামের বানান দেখুন। `state lock` হলে অন্য প্রসেস দেখুন, তারপর `force-unlock`। ক্লাউডে `AccessDenied` হলে ক্রেডেনশিয়াল ও IAM পরীক্ষা করুন।

পরিষ্কার: `terraform destroy -auto-approve` চালিয়ে `Destroy complete!` দেখে থামুন; ক্লাউডে destroy না করলে বিল চলবে। `.gitignore`-এ `.terraform/`, `*.tfstate`, `*.tfvars` রাখুন — স্টেট বা সিক্রেট কখনো Git-এ নয়। **পর্যালোচনা:** ১) `plan` ও `apply`-এর পার্থক্য কী? ২) স্টেট ফাইল মুছে গেলে কী হবে? ৩) এই কোড প্রোডাকশনে ব্যবহার করতে কী বদলাতে হবে?

Lab 16 • Volume 08

Lab 16 — Import Existing Resources

এই ল্যাবে **বিদ্যমান রিসোর্স ইমপোর্ট** হাতে-কলমে শিখবেন (TechNova, `technova.example`, `203.0.113.x`)। প্রতিটি কমান্ডের আউটপুট নিজে পড়ুন।

১৬.১ উদ্দেশ্য, টুলস ও পূর্বশর্ত

ল্যাব শেষে পারবেন: বিদ্যমান রিসোর্স ইমপোর্ট–এর ধারণা ব্যাখ্যা করতে, ফাইল লিখে `init`, `plan` ও `apply` চালাতে, এবং আউটপুট ও স্টেট পড়ে ফল যাচাই করতে। প্রয়োজন: Ubuntu 24.04, Terraform >= 1.9, < 2.0, Git। আলাদা ডিরেক্টরি ব্যবহার করুন: `mkdir -p ~/labs/lab16 && cd ~/labs/lab16`। ক্লাউড ল্যাবে ক্রেডেনশিয়াল পরিবেশ ভ্যারিয়েবলে সেট করুন, কোডে নয়।

১৬.২ আর্কিটেকচার

```
You (Ubuntu 24.04, Terraform 1.9+) --init/plan/apply--> main.tf --> Provider --> terraform.tfst
```

১৬.৩ ধাপে ধাপে নির্দেশনা

ধাপ ১: ডিরেক্টরি তৈরি করুন। **ধাপ ২:** নিচের `main.tf` লিখুন। **ধাপ ৩:** `terraform fmt` ও `terraform validate` চালান। **ধাপ ৪:** `plan` পড়ে নিশ্চিত হয়ে `apply` দিন।

```
resource "local_file" "imported" {
  content  = "imported-content"
  filename = "${path.module}/imported.txt"
}
```

```
echo "imported-content" > imported.txt
terraform import local_file.imported imported.txt
terraform plan
terraform apply -auto-approve
```

> **খরচ:** সম্পূর্ণ লোকাল — কোনো ক্লাউড খরচ নেই।

১৬.৪ যাচাই, সমস্যা সমাধান ও পরিষ্কার

সফল হলে `Apply complete!` দেখবেন। যাচাই: `terraform state list; terraform output; terraform plan` — দ্বিতীয় plan–এ `No changes` আসা উচিত।

> **TROUBLESHOOTING:** `command not found` হলে Terraform আবার ইনস্টল করুন (Lab 1)। `Invalid reference` হলে নামের বানান দেখুন। `state lock` হলে অন্য প্রসেস দেখুন, তারপর `force-unlock`। ক্লাউডে `AccessDenied` হলে ক্রেডেনশিয়াল ও IAM পরীক্ষা করুন।

পরিষ্কার: `terraform destroy -auto-approve` চালিয়ে `Destroy complete!` দেখে থামুন; ক্লাউডে destroy না করলে বিল চলবে। `.gitignore`-এ `.terraform/`, `*.tfstate`, `*.tfvars` রাখুন — স্টেট বা সিক্রেট কখনো Git-এ নয়। **পর্যালোচনা:** ১) `plan` ও `apply`-এর পার্থক্য কী? ২) স্টেট ফাইল মুছে গেলে কী হবে? ৩) এই কোড প্রোডাকশনে ব্যবহার করতে কী বদলাতে হবে?

Lab 17 • Volume 08

Lab 17 — AWS Infrastructure

এই ল্যাবে **AWS ইনফ্রাস্ট্রাকচার** হাতে-কলমে শিখবেন (TechNova, `technova.example`, `203.0.113.x`)। প্রতিটি কমান্ডের আউটপুট নিজে পড়ুন।

১৭.১ উদ্দেশ্য, টুলস ও পূর্বশর্ত

ল্যাব শেষে পারবেন: AWS ইনফ্রাস্ট্রাকচার–এর ধারণা ব্যাখ্যা করতে, ফাইল লিখে `init`, `plan` ও `apply` চালাতে, এবং আউটপুট ও স্টেট পড়ে ফল যাচাই করতে। প্রয়োজন: Ubuntu 24.04, Terraform >= 1.9, < 2.0, Git। আলাদা ডিরেক্টরি ব্যবহার করুন: `mkdir -p ~/labs/lab17 && cd ~/labs/lab17`। ক্লাউড ল্যাবে ক্রেডেনশিয়াল পরিবেশ ভ্যারিয়েবলে সেট করুন, কোডে নয়।

১৭.২ আর্কিটেকচার

```
You (Ubuntu 24.04, Terraform 1.9+) --init/plan/apply--> main.tf --> Provider --> terraform.tfst
```

১৭.৩ ধাপে ধাপে নির্দেশনা

ধাপ ১: ডিরেক্টরি তৈরি করুন। **ধাপ ২:** নিচের `main.tf` লিখুন। **ধাপ ৩:** `terraform fmt` ও `terraform validate` চালান। **ধাপ ৪:** `plan` পড়ে নিশ্চিত হয়ে `apply` দিন।

```
terraform {
  required_version = ">= 1.9, < 2.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.region
}

variable "region" {
  type    = string
  default = "ap-south-1"
}

resource "aws_vpc" "main" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_hostnames = true
  tags = { Name = "technova-vpc" }
}
```

```
resource "aws_subnet" "public" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.0.1.0/24"
  map_public_ip_on_launch = true
  tags = { Name = "technova-public" }
}
```

```
terraform init
terraform plan
terraform apply -auto-approve
terraform destroy -auto-approve
```

WARNING: ক্লাউড খরচ হবে — বাজেট অ্যালার্ট সেট করুন, ছোট সাইজ ব্যবহার করুন, শেষে `terraform destroy` চালান।

১৭.৪ যাচাই, সমস্যা সমাধান ও পরিষ্কার

সফল হলে `Apply complete!` দেখবেন। যাচাই: `terraform state list; terraform output; terraform plan` — দ্বিতীয় plan-এ `No changes` আসা উচিত।

TROUBLESHOOTING: `command not found` হলে Terraform আবার ইনস্টল করুন (Lab 1)। `Invalid reference` হলে নামের বানান দেখুন। `state lock` হলে অন্য প্রসেস দেখুন, তারপর `force-unlock`। ক্লাউডে `AccessDenied` হলে ক্রেডেনশিয়াল ও IAM পরীক্ষা করুন।

পরিষ্কার: `terraform destroy -auto-approve` চালিয়ে `Destroy complete!` দেখে থামুন; ক্লাউডে destroy না করলে বিল চলবে। `.gitignore`-এ `.terraform/`, `*.tfstate`, `*.tfvars` রাখুন — স্টেট বা সিক্রেট কখনো Git-এ নয়। **পর্যালোচনা:** ১) `plan` ও `apply`-এর পার্থক্য কী? ২) স্টেট ফাইল মুছে গেলে কী হবে? ৩) এই কোড প্রোডাকশনে ব্যবহার করতে কী বদলাতে হবে?

Lab 18 • Volume 08

Lab 18 — Azure Infrastructure

এই ল্যাবে **Azure ইনফ্রাস্ট্রাকচার** হাতে-কলমে শিখবেন (TechNova, `technova.example`, `203.0.113.x`)। প্রতিটি কমান্ডের আউটপুট নিজে পড়ুন।

১৮.১ উদ্দেশ্য, টুলস ও পূর্বশর্ত

ল্যাব শেষে পারবেন: Azure ইনফ্রাস্ট্রাকচার-এর ধারণা ব্যাখ্যা করতে, ফাইল লিখে `init`, `plan` ও `apply` চালাতে, এবং আউটপুট ও স্টেট পড়ে ফল যাচাই করতে। প্রয়োজন: Ubuntu 24.04, Terraform >= 1.9, < 2.0, Git। আলাদা ডিরেক্টরি ব্যবহার করুন: `mkdir -p ~/labs/lab18 && cd ~/labs/lab18`। ক্লাউড ল্যাবে ক্রেডেনশিয়াল পরিবেশ ভ্যারিয়েবলে সেট করুন, কোডে নয়।

১৮.২ আর্কিটেকচার

```
You (Ubuntu 24.04, Terraform 1.9+) --init/plan/apply--> main.tf --> Provider --> terraform.tfst
```

১৮.৩ ধাপে ধাপে নির্দেশনা

ধাপ ১: ডিরেক্টরি তৈরি করুন। **ধাপ ২:** নিচের `main.tf` লিখুন। **ধাপ ৩:** `terraform fmt` ও `terraform validate` চালান। **ধাপ ৪:** `plan` পড়ে নিশ্চিত হয়ে `apply` দিন।

```
terraform {
  required_version = ">= 1.9, < 2.0"
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 4.0"
    }
  }
}

provider "azurerm" {
  features {}
}

resource "azurerm_resource_group" "main" {
  name     = "technova-rg"
  location = "Southeast Asia"
}

resource "azurerm_virtual_network" "main" {
  name                = "technova-vnet"
  address_space       = ["10.1.0.0/16"]
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
}
```

```
terraform init
terraform plan
terraform apply -auto-approve
terraform destroy -auto-approve
```

> **WARNING:** ক্লাউড খরচ হবে — বাজেট অ্যালার্ট সেট করুন, ছোট সাইজ ব্যবহার করুন, শেষে `terraform destroy` চালান।

১৮.৪ যাচাই, সমস্যা সমাধান ও পরিষ্কার

সফল হলে `Apply complete!` দেখবেন। যাচাই: `terraform state list; terraform output; terraform plan` — দ্বিতীয় plan-এ `No changes` আসা উচিত।

> **TROUBLESHOOTING:** `command not found` হলে Terraform আবার ইনস্টল করুন (Lab 1)। `Invalid reference` হলে নামের বানান দেখুন। `state lock` হলে অন্য প্রসেস দেখুন, তারপর `force-unlock`। ক্লাউডে `AccessDenied` হলে ক্রেডেনশিয়াল ও IAM পরীক্ষা করুন।

পরিষ্কার: `terraform destroy -auto-approve` চালিয়ে `Destroy complete!` দেখে থামুন; ক্লাউডে destroy না করলে বিল চলবে। `.gitignore`-এ `.terraform/`, `*.tfstate`, `*.tfvars` রাখুন — স্টেট বা সিক্রেট কখনো Git-এ নয়। **পর্যালোচনা:** ১) `plan` ও `apply`-এর পার্থক্য কী? ২) স্টেট ফাইল মুছে গেলে কী হবে? ৩) এই কোড প্রোডাকশনে ব্যবহার করতে কী বদলাতে হবে?

Lab 19 • Volume 08

Lab 19 — GCP Infrastructure

এই ল্যাবে **GCP ইনফ্রাস্ট্রাকচার** হাতে-কলমে শিখবেন (TechNova, `technova.example`, `203.0.113.x`)। প্রতিটি কমান্ডের আউটপুট নিজে পড়ুন।

১৯.১ উদ্দেশ্য, টুলস ও পূর্বশর্ত

ল্যাব শেষে পারবেন: GCP ইনফ্রাস্ট্রাকচার–এর ধারণা ব্যাখ্যা করতে, ফাইল লিখে `init`, `plan` ও `apply` চালাতে, এবং আউটপুট ও স্টেট পড়ে ফল যাচাই করতে। প্রয়োজন: Ubuntu 24.04, Terraform >= 1.9, < 2.0, Git। আলাদা ডিরেক্টরি ব্যবহার করুন: `mkdir -p ~/labs/lab19 && cd ~/labs/lab19`। ক্লাউড ল্যাবে ক্রেডেনশিয়াল পরিবেশ ভ্যারিয়েবলে সেট করুন, কোডে নয়।

১৯.২ আর্কিটেকচার

```
You (Ubuntu 24.04, Terraform 1.9+) --init/plan/apply--> main.tf --> Provider --> terraform.tfst
```

১৯.৩ ধাপে ধাপে নির্দেশনা

ধাপ ১: ডিরেক্টরি তৈরি করুন। **ধাপ ২:** নিচের `main.tf` লিখুন। **ধাপ ৩:** `terraform fmt` ও `terraform validate` চালান। **ধাপ ৪:** `plan` পড়ে নিশ্চিত হয়ে `apply` দিন।

```
terraform {
  required_version = ">= 1.9, < 2.0"
  required_providers {
    google = {
      source  = "hashicorp/google"
      version = "~> 6.0"
    }
  }
}

provider "google" {
  project = var.project_id
  region  = "asia-south1"
}

variable "project_id" {
  type = string
}

resource "google_compute_network" "main" {
  name                    = "technova-vpc"
  auto_create_subnetworks = false
}

resource "google_compute_subnetwork" "main" {
```

```
  name          = "technova-subnet"
  ip_cidr_range = "10.2.0.0/24"
  region        = "asia-south1"
  network       = google_compute_network.main.id
}
```

```
terraform init
terraform plan
terraform apply -auto-approve
terraform destroy -auto-approve
```

> **WARNING:** ক্লাউড খরচ হবে — বাজেট অ্যালার্ট সেট করুন, ছোট সাইজ ব্যবহার করুন, শেষে `terraform destroy` চালান।

১৯.৪ যাচাই, সমস্যা সমাধান ও পরিষ্কার

সফল হলে `Apply complete!` দেখবেন। যাচাই: `terraform state list; terraform output; terraform plan` — দ্বিতীয় plan-এ `No changes` আসা উচিত।

> **TROUBLESHOOTING:** `command not found` হলে Terraform আবার ইনস্টল করুন (Lab 1)। `Invalid reference` হলে নামের বানান দেখুন। `state lock` হলে অন্য প্রসেস দেখুন, তারপর `force-unlock`। ক্লাউডে `AccessDenied` হলে ক্রেডেনশিয়াল ও IAM পরীক্ষা করুন।

পরিষ্কার: `terraform destroy -auto-approve` চালিয়ে `Destroy complete!` দেখে থামুন; ক্লাউডে destroy না করলে বিল চলবে। `.gitignore`-এ `.terraform/`, `*.tfstate`, `*.tfvars` রাখুন — স্টেট বা সিক্রেট কখনো Git-এ নয়। **পর্যালোচনা:** ১) `plan` ও `apply`-এর পার্থক্য কী? ২) স্টেট ফাইল মুছে গেলে কী হবে? ৩) এই কোড প্রোডাকশনে ব্যবহার করতে কী বদলাতে হবে?

Lab 20 • Volume 08

Lab 20 — IBM Cloud Infrastructure

এই ল্যাবে **IBM Cloud ইনফ্রাস্ট্রাকচার** হাতে-কলমে শিখবেন (TechNova, `technova.example`, `203.0.113.x`)। প্রতিটি কমান্ডের আউটপুট নিজে পড়ুন।

২০.১ উদ্দেশ্য, টুলস ও পূর্বশর্ত

ল্যাব শেষে পারবেন: IBM Cloud ইনফ্রাস্ট্রাকচার–এর ধারণা ব্যাখ্যা করতে, ফাইল লিখে `init`, `plan` ও `apply` চালাতে, এবং আউটপুট ও স্টেট পড়ে ফল যাচাই করতে। প্রয়োজন: Ubuntu 24.04, Terraform >= 1.9, < 2.0, Git। আলাদা ডিরেক্টরি ব্যবহার করুন: `mkdir -p ~/labs/lab20 && cd ~/labs/lab20`। ক্লাউড ল্যাবে ক্রেডেনশিয়াল পরিবেশ ভ্যারিয়েবলে সেট করুন, কোডে নয়।

২০.২ আর্কিটেকচার

```
You (Ubuntu 24.04, Terraform 1.9+) --init/plan/apply--> main.tf --> Provider --> terraform.tfst
```

২০.৩ ধাপে ধাপে নির্দেশনা

ধাপ ১: ডিরেক্টরি তৈরি করুন। **ধাপ ২:** নিচের `main.tf` লিখুন। **ধাপ ৩:** `terraform fmt` ও `terraform validate` চালান। **ধাপ ৪:** `plan` পড়ে নিশ্চিত হয়ে `apply` দিন।

```
terraform {
  required_version = ">= 1.9, < 2.0"
  required_providers {
    ibm = {
      source  = "IBM-Cloud/ibm"
      version = "~> 1.0"
    }
  }
}

provider "ibm" {
  ibmcloud_api_key = var.ibmcloud_api_key
  region           = "eu-gb"
}

variable "ibmcloud_api_key" {
  type      = string
  sensitive = true
}

resource "ibm_resource_group" "main" {
  name = "technova-rg"
}

resource "ibm_is_vpc" "main" {
```

```
  name           = "technova-vpc"
  resource_group = ibm_resource_group.main.id
}
```

```
terraform init
terraform plan
terraform apply -auto-approve
terraform destroy -auto-approve
```

WARNING: ক্লাউড খরচ হবে — বাজেট অ্যালার্ট সেট করুন, ছোট সাইজ ব্যবহার করুন, শেষে `terraform destroy` চালান।

২০.৪ যাচাই, সমস্যা সমাধান ও পরিষ্কার

সফল হলে `Apply complete!` দেখবেন। যাচাই: `terraform state list; terraform output; terraform plan` — দ্বিতীয় plan-এ `No changes` আসা উচিত।

TROUBLESHOOTING: `command not found` হলে Terraform আবার ইনস্টল করুন (Lab 1)। `Invalid reference` হলে নামের বানান দেখুন। `state lock` হলে অন্য প্রসেস দেখুন, তারপর `force-unlock`। ক্লাউডে `AccessDenied` হলে ক্রেডেনশিয়াল ও IAM পরীক্ষা করুন।

পরিষ্কার: `terraform destroy -auto-approve` চালিয়ে `Destroy complete!` দেখে থামুন; ক্লাউডে destroy না করলে বিল চলবে। `.gitignore`-এ `.terraform/`, `*.tfstate`, `*.tfvars` রাখুন — স্টেট বা সিক্রেট কখনো Git-এ নয়। **পর্যালোচনা:** ১) `plan` ও `apply`-এর পার্থক্য কী? ২) স্টেট ফাইল মুছে গেলে কী হবে? ৩) এই কোড প্রোডাকশনে ব্যবহার করতে কী বদলাতে হবে?

Lab 21 • Volume 08

Lab 21 — IAM and Network Automation

এই ল্যাবে **IAM ও নেটওয়ার্ক অটোমেশন** হাতে-কলমে শিখবেন (TechNova, `technova.example`, `203.0.113.x`)। প্রতিটি কমান্ডের আউটপুট নিজে পড়ুন।

২১.১ উদ্দেশ্য, টুলস ও পূর্বশর্ত

ল্যাব শেষে পারবেন: IAM ও নেটওয়ার্ক অটোমেশন–এর ধারণা ব্যাখ্যা করতে, ফাইল লিখে `init`, `plan` ও `apply` চালাতে, এবং আউটপুট ও স্টেট পড়ে ফল যাচাই করতে। প্রয়োজন: Ubuntu 24.04, Terraform >= 1.9, < 2.0, Git। আলাদা ডিরেক্টরি ব্যবহার করুন: `mkdir -p ~/labs/lab21 && cd ~/labs/lab21`। ক্লাউড ল্যাবে ক্রেডেনশিয়াল পরিবেশ ভ্যারিয়েবলে সেট করুন, কোডে নয়।

২১.২ আর্কিটেকচার

```
You (Ubuntu 24.04, Terraform 1.9+) --init/plan/apply--> main.tf --> Provider --> terraform.tfst
```

২১.৩ ধাপে ধাপে নির্দেশনা

ধাপ ১: ডিরেক্টরি তৈরি করুন। **ধাপ ২:** নিচের `main.tf` লিখুন। **ধাপ ৩:** `terraform fmt` ও `terraform validate` চালান। **ধাপ ৪:** `plan` পড়ে নিশ্চিত হয়ে `apply` দিন।

```
resource "aws_iam_role" "app" {
  name = "technova-app-role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect    = "Allow"
      Principal = { Service = "ec2.amazonaws.com" }
      Action    = "sts:AssumeRole"
    }]
  })
}

resource "aws_security_group" "web" {
  name   = "technova-web-sg"
  vpc_id = aws_vpc.main.id
  ingress {
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["203.0.113.0/24"]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
```

```
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

```
terraform init
terraform plan
terraform apply -auto-approve
terraform destroy -auto-approve
```

> **WARNING:** ক্লাউড খরচ হবে — বাজেট অ্যালার্ট সেট করুন, ছোট সাইজ ব্যবহার করুন, শেষে `terraform destroy` চালান।

২১.৪ যাচাই, সমস্যা সমাধান ও পরিষ্কার

সফল হলে `Apply complete!` দেখবেন। যাচাই: `terraform state list; terraform output; terraform plan` — দ্বিতীয় plan-এ `No changes` আসা উচিত।

> **TROUBLESHOOTING:** `command not found` হলে Terraform আবার ইনস্টল করুন (Lab 1)। `Invalid reference` হলে নামের বানান দেখুন। `state lock` হলে অন্য প্রসেস দেখুন, তারপর `force-unlock`। ক্লাউডে `AccessDenied` হলে ক্রেডেনশিয়াল ও IAM পরীক্ষা করুন।

পরিষ্কার: `terraform destroy -auto-approve` চালিয়ে `Destroy complete!` দেখে থামুন; ক্লাউডে destroy না করলে বিল চলবে। `.gitignore`-এ `.terraform/`, `*.tfstate`, `*.tfvars` রাখুন — স্টেট বা সিক্রেট কখনো Git-এ নয়। **পর্যালোচনা:** ১) `plan` ও `apply`-এর পার্থক্য কী? ২) স্টেট ফাইল মুছে গেলে কী হবে? ৩) এই কোড প্রোডাকশনে ব্যবহার করতে কী বদলাতে হবে?

Lab 22 • Volume 08

Lab 22 — Validation and Security Scanning

এই ল্যাবে **ভ্যালিডেশন ও নিরাপত্তা স্ক্যানিং** হাতে-কলমে শিখবেন (TechNova, `technova.example`, `203.0.113.x`)। প্রতিটি কমান্ডের আউটপুট নিজে পড়ুন।

২২.১ উদ্দেশ্য, টুলস ও পূর্বশর্ত

ল্যাব শেষে পারবেন: ভ্যালিডেশন ও নিরাপত্তা স্ক্যানিং-এর ধারণা ব্যাখ্যা করতে, ফাইল লিখে `init`, `plan` ও `apply` চালাতে, এবং আউটপুট ও স্টেট পড়ে ফল যাচাই করতে। প্রয়োজন: Ubuntu 24.04, Terraform >= 1.9, < 2.0, Git। আলাদা ডিরেক্টরি ব্যবহার করুন: `mkdir -p ~/labs/lab22 && cd ~/labs/lab22`। ক্লাউড ল্যাবে ক্রেডেনশিয়াল পরিবেশ ভ্যারিয়েবলে সেট করুন, কোডে নয়।

২২.২ আর্কিটেকচার

```
You (Ubuntu 24.04, Terraform 1.9+) --init/plan/apply--> main.tf --> Provider --> terraform.tfst
```

২২.৩ ধাপে ধাপে নির্দেশনা

ধাপ ১: ডিরেক্টরি তৈরি করুন। **ধাপ ২:** নিচের `main.tf` লিখুন। **ধাপ ৩:** `terraform fmt` ও `terraform validate` চালান। **ধাপ ৪:** `plan` পড়ে নিশ্চিত হয়ে `apply` দিন।

```
terraform {
  required_version = ">= 1.9, < 2.0"
}

variable "environment" {
  type = string
  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "Environment must be dev, staging or prod."
  }
}
```

```
terraform fmt -check -recursive
terraform validate
tflint --init && tflint
checkov -d . --framework terraform
tfsec .
```

> **খরচ:** সম্পূর্ণ লোকাল — কোনো ক্লাউড খরচ নেই।

২২.৪ যাচাই, সমস্যা সমাধান ও পরিষ্কার

সফল হলে `Apply complete!` দেখবেন। যাচাই: `terraform state list; terraform output; terraform plan` — দ্বিতীয় plan-এ `No changes` আসা উচিত।

> **TROUBLESHOOTING:** `command not found` হলে Terraform আবার ইনস্টল করুন (Lab 1)। `Invalid reference` হলে নামের বানান দেখুন। `state lock` হলে অন্য প্রসেস দেখুন, তারপর `force-unlock`। ক্লাউডে `AccessDenied` হলে ক্রেডেনশিয়াল ও IAM পরীক্ষা করুন।

পরিষ্কার: `terraform destroy -auto-approve` চালিয়ে `Destroy complete!` দেখে থামুন; ক্লাউডে destroy না করলে বিল চলবে। `.gitignore`-এ `.terraform/`, `*.tfstate`, `*.tfvars` রাখুন — স্টেট বা সিক্রেট কখনো Git-এ নয়। **পর্যালোচনা:** ১) `plan` ও `apply`-এর পার্থক্য কী? ২) স্টেট ফাইল মুছে গেলে কী হবে? ৩) এই কোড প্রোডাকশনে ব্যবহার করতে কী বদলাতে হবে?

Lab 23 • Volume 08

Lab 23 — Terraform CI/CD Pipeline

এই ল্যাবে **Terraform CI/CD পাইপলাইন** হাতে-কলমে শিখবেন (TechNova, `technova.example`, `203.0.113.x`)। প্রতিটি কমান্ডের আউটপুট নিজে পড়ুন।

২৩.১ উদ্দেশ্য, টুলস ও পূর্বশর্ত

ল্যাব শেষে পারবেন: Terraform CI/CD পাইপলাইন–এর ধারণা ব্যাখ্যা করতে, ফাইল লিখে `init`, `plan` ও `apply` চালাতে, এবং আউটপুট ও স্টেট পড়ে ফল যাচাই করতে। প্রয়োজন: Ubuntu 24.04, Terraform >= 1.9, < 2.0, Git। আলাদা ডিরেক্টরি ব্যবহার করুন: `mkdir -p ~/labs/lab23 && cd ~/labs/lab23`। ক্লাউড ল্যাবে ক্রেডেনশিয়াল পরিবেশ ভ্যারিয়েবলে সেট করুন, কোডে নয়।

২৩.২ আর্কিটেকচার

```
You (Ubuntu 24.04, Terraform 1.9+) --init/plan/apply--> main.tf --> Provider --> terraform.tfst
```

২৩.৩ ধাপে ধাপে নির্দেশনা

ধাপ ১: ডিরেক্টরি তৈরি করুন। **ধাপ ২:** নিচের `main.tf` লিখুন। **ধাপ ৩:** `terraform fmt` ও `terraform validate` চালান। **ধাপ ৪:** `plan` পড়ে নিশ্চিত হয়ে `apply` দিন।

```
# .github/workflows/terraform.yml
name: Terraform
on:
  pull_request:
    paths: ["terraform/**"]
  push:
    branches: ["main"]
jobs:
  terraform:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: "1.9.8"
      - run: terraform fmt -check -recursive
      - run: terraform init -backend=false
      - run: terraform validate
      - run: terraform plan -no-color
```

```
git add . && git commit -m "Add Terraform CI"
git push origin main
# Open a pull request and read the plan output
```

> **খরচ:** সম্পূর্ণ লোকাল — কোনো ক্লাউড খরচ নেই।

২৩.৪ যাচাই, সমস্যা সমাধান ও পরিষ্কার

সফল হলে `Apply complete!` দেখবেন। যাচাই: `terraform state list; terraform output; terraform plan` — দ্বিতীয় plan-এ `No changes` আসা উচিত।

> **TROUBLESHOOTING:** `command not found` হলে Terraform আবার ইনস্টল করুন (Lab 1)। `Invalid reference` হলে নামের বানান দেখুন। `state lock` হলে অন্য প্রসেস দেখুন, তারপর `force-unlock`। ক্লাউডে `AccessDenied` হলে ক্রেডেনশিয়াল ও IAM পরীক্ষা করুন।

পরিষ্কার: `terraform destroy -auto-approve` চালিয়ে `Destroy complete!` দেখে থামুন; ক্লাউডে destroy না করলে বিল চলবে। `.gitignore`-এ `.terraform/`, `*.tfstate`, `*.tfvars` রাখুন — স্টেট বা সিক্রেট কখনো Git-এ নয়। **পর্যালোচনা:** ১) `plan` ও `apply`-এর পার্থক্য কী? ২) স্টেট ফাইল মুছে গেলে কী হবে? ৩) এই কোড প্রোডাকশনে ব্যবহার করতে কী বদলাতে হবে?

Lab 24 • Volume 08

Lab 24 — Complete Production-Style Infrastructure Deployment

এই ল্যাবে **সম্পূর্ণ প্রোডাকশন-ধাঁচের ইনফ্রাস্ট্রাকচার ডেপ্লয়মেন্ট** হাতে-কলমে শিখবেন (TechNova, `technova.example`, `203.0.113.x`)। প্রতিটি কমান্ডের আউটপুট নিজে পড়ুন।

২৪.১ উদ্দেশ্য, টুলস ও পূর্বশর্ত

ল্যাব শেষে পারবেন: সম্পূর্ণ প্রোডাকশন-ধাঁচের ইনফ্রাস্ট্রাকচার ডেপ্লয়মেন্ট–এর ধারণা ব্যাখ্যা করতে, ফাইল লিখে `init`, `plan` ও `apply` চালাতে, এবং আউটপুট ও স্টেট পড়ে ফল যাচাই করতে। প্রয়োজন: Ubuntu 24.04, Terraform >= 1.9, < 2.0, Git। আলাদা ডিরেক্টরি ব্যবহার করুন: `mkdir -p ~/labs/lab24 && cd ~/labs/lab24`। ক্লাউড ল্যাবে ক্রেডেনশিয়াল পরিবেশ ভ্যারিয়েবলে সেট করুন, কোডে নয়।

২৪.২ আর্কিটেকচার

```
You (Ubuntu 24.04, Terraform 1.9+) --init/plan/apply--> main.tf --> Provider --> terraform.tfst
```

২৪.৩ ধাপে ধাপে নির্দেশনা

ধাপ ১: ডিরেক্টরি তৈরি করুন। **ধাপ ২:** নিচের `main.tf` লিখুন। **ধাপ ৩:** `terraform fmt` ও `terraform validate` চালান। **ধাপ ৪:** `plan` পড়ে নিশ্চিত হয়ে `apply` দিন।

```
module "network" {
  source              = "./modules/network"
  cidr_block          = "10.0.0.0/16"
  public_subnet_cidrs = ["10.0.1.0/24", "10.0.2.0/24"]
}

module "compute" {
  source     = "./modules/compute"
  vpc_id     = module.network.vpc_id
  subnet_ids = module.network.public_subnet_ids
  image_id   = "ami-CHANGE-ME"
  instance_type = "t3.micro"
}

module "storage" {
  source = "./modules/storage"
  bucket = "CHANGE-ME-technova-prod-assets"
}
```

```
terraform init
terraform plan -out=tfplan-prod
```

```
terraform apply tfplan-prod
terraform destroy -auto-approve
```

> **WARNING:** ক্লাউড খরচ হবে — বাজেট অ্যালার্ট সেট করুন, ছোট সাইজ ব্যবহার করুন, শেষে `terraform destroy` চালান।

২৪.৪ যাচাই, সমস্যা সমাধান ও পরিষ্কার

সফল হলে `Apply complete!` দেখবেন। যাচাই: `terraform state list; terraform output; terraform plan` — দ্বিতীয় plan-এ `No changes` আসা উচিত।

> **TROUBLESHOOTING:** `command not found` হলে Terraform আবার ইনস্টল করুন (Lab 1)। `Invalid reference` হলে নামের বানান দেখুন। `state lock` হলে অন্য প্রসেস দেখুন, তারপর `force-unlock`। ক্লাউডে `AccessDenied` হলে ক্রেডেনশিয়াল ও IAM পরীক্ষা করুন।

পরিষ্কার: `terraform destroy -auto-approve` চালিয়ে `Destroy complete!` দেখে থামুন; ক্লাউডে destroy না করলে বিল চলবে। `.gitignore`-এ `.terraform/`, `*.tfstate`, `*.tfvars` রাখুন — স্টেট বা সিক্রেট কখনো Git-এ নয়। **পর্যালোচনা:** ১) `plan` ও `apply`-এর পার্থক্য কী? ২) স্টেট ফাইল মুছে গেলে কী হবে? ৩) এই কোড প্রোডাকশনে ব্যবহার করতে কী বদলাতে হবে?

Assignment 1 • Volume 08

Assignment 1 — Reusable VPC Module

১.১ উদ্দেশ্য

পুনর্ব্যবহারযোগ্য VPC মডিউল: VPC, দুটি পাবলিক ও দুটি প্রাইভেট সাবনেট, NAT, রুট টেবিল; আউটপুটে VPC ID ও সাবনেট ID। সময়: ২–৪ ঘণ্টা। TechNova (`technova.example`) পরিবেশে কাজ করুন।

১.২ ডেলিভারেবল ও গ্রহণযোগ্যতা

ডেলিভারেবল: Terraform কোড (`main.tf`, `variables.tf`, `outputs.tf`, `versions.tf`, `README.md`), এক পৃষ্ঠার পরিকল্পনা নোট, `plan` / `apply` প্রমাণ, ক্লাউড কাজে `destroy` প্রমাণ। মানদণ্ড: `fmt -check` ও `validate` পাস; plan–এ অপ্রত্যাশিত destroy নেই; কোনো সিক্রেট বা `*.tfstate` Git–এ নেই; দ্বিতীয় plan–এ `No changes`। ইঙ্গিত: একটি রিসোর্স দিয়ে শুরু করে plan দেখুন, তারপর বাড়ান।

Assignment 2 • Volume 08

Assignment 2 — Local to Remote State Migration

২.১ উদ্দেশ্য

লোকাল স্টেট রিমোট ব্যাকএন্ডে মাইগ্রেট: S3-সদৃশ রিমোট ব্যাকএন্ডে স্টেট সরানো, লকিং চালু, মাইগ্রেশনের আগে ব্যাকআপ। সময়: ২–৪ ঘণ্টা। TechNova (`technova.example`) পরিবেশে কাজ করুন।

২.২ ডেলিভারেবল ও গ্রহণযোগ্যতা

ডেলিভারেবল: Terraform কোড (`main.tf`, `variables.tf`, `outputs.tf`, `versions.tf`, `README.md`), এক পৃষ্ঠার পরিকল্পনা নোট, `plan` / `apply` প্রমাণ, ক্লাউড কাজে `destroy` প্রমাণ। মানদণ্ড: `fmt -check` ও `validate` পাস; plan-এ অপ্রত্যাশিত destroy নেই; কোনো সিক্রেট বা `*.tfstate` Git-এ নেই; দ্বিতীয় plan-এ `No changes`। ইঙ্গিত: একটি রিসোর্স দিয়ে শুরু করে plan দেখুন, তারপর বাড়ান।

Assignment 3 • Volume 08

Assignment 3 — Module with Version Constraints

৩.১ উদ্দেশ্য

সংস্করণ সীমাসহ মডিউল: Terraform ও প্রোভাইডার সংস্করণ পিন করা, `required_version` ব্যবহার। সময়: ২–৪ ঘণ্টা। TechNova (`technova.example`) পরিবেশে কাজ করুন।

৩.২ ডেলিভারেবল ও গ্রহণযোগ্যতা

ডেলিভারেবল: Terraform কোড (`main.tf`, `variables.tf`, `outputs.tf`, `versions.tf`, `README.md`), এক পৃষ্ঠার পরিকল্পনা নোট, `plan` / `apply` প্রমাণ, ক্লাউড কাজে `destroy` প্রমাণ। মানদণ্ড: `fmt -check` ও `validate` পাস; plan–এ অপ্রত্যাশিত destroy নেই; কোনো সিক্রেট বা `*.tfstate` Git–এ নেই; দ্বিতীয় plan–এ `No changes`। ইঙ্গিত: একটি রিসোর্স দিয়ে শুরু করে plan দেখুন, তারপর বাড়ান।

Assignment 4 • Volume 08

Assignment 4 — Import a Manually Created Resource

৪.১ উদ্দেশ্য

হাতে তৈরি রিসোর্স ইমপোর্ট: কনসোলে তৈরি রিসোর্স `terraform import` দিয়ে স্টেটে আনা, plan–এ No changes আনা। সময়: ২–৪ ঘণ্টা। TechNova (`technova.example`) পরিবেশে কাজ করুন।

৪.২ ডেলিভারেবল ও গ্রহণযোগ্যতা

ডেলিভারেবল: Terraform কোড (`main.tf` , `variables.tf` , `outputs.tf` , `versions.tf` , `README.md`), এক পৃষ্ঠার পরিকল্পনা নোট, `plan` / `apply` প্রমাণ, ক্লাউড কাজে `destroy` প্রমাণ। মানদণ্ড: `fmt -check` ও `validate` পাস; plan–এ অপ্রত্যাশিত destroy নেই; কোনো সিক্রেট বা `*.tfstate` Git–এ নেই; দ্বিতীয় plan–এ `No changes` । ইঙ্গিত: একটি রিসোর্স দিয়ে শুরু করে plan দেখুন, তারপর বাড়ান।

Assignment 5 • Volume 08

Assignment 5 — Multi-Environment Layout

৫.১ উদ্দেশ্য

মাল্টি-এনভায়রনমেন্ট লেআউট: dev/staging/prod আলাদা ভ্যারিয়েবল ফাইল, আলাদা স্টেট, একই মডিউল। সময়: ২–৪ ঘণ্টা। TechNova (`technova.example`) পরিবেশে কাজ করুন।

৫.২ ডেলিভারেবল ও গ্রহণযোগ্যতা

ডেলিভারেবল: Terraform কোড (`main.tf` , `variables.tf` , `outputs.tf` , `versions.tf` , `README.md`), এক পৃষ্ঠার পরিকল্পনা নোট, `plan` / `apply` প্রমাণ, ক্লাউড কাজে `destroy` প্রমাণ। মানদণ্ড: `fmt -check` ও `validate` পাস; plan–এ অপ্রত্যাশিত destroy নেই; কোনো সিক্রেট বা `*.tfstate` Git–এ নেই; দ্বিতীয় plan–এ `No changes` । ইঙ্গিত: একটি রিসোর্স দিয়ে শুরু করে plan দেখুন, তারপর বাড়ান।

Assignment 6 • Volume 08

Assignment 6 — Policy-as-Code Check

৬.১ উদ্দেশ্য

পলিসি-অ্যাজ-কোড চেক: ট্যাগ বাধ্যতামূলক, পাবলিক S3 নিষিদ্ধ, `checkov` বা OPA দিয়ে যাচাই। সময়: ২–৪ ঘণ্টা। TechNova (`technova.example`) পরিবেশে কাজ করুন।

৬.২ ডেলিভারেবল ও গ্রহণযোগ্যতা

ডেলিভারেবল: Terraform কোড (`main.tf`, `variables.tf`, `outputs.tf`, `versions.tf`, `README.md`), এক পৃষ্ঠার পরিকল্পনা নোট, `plan` / `apply` প্রমাণ, ক্লাউড কাজে `destroy` প্রমাণ। মানদণ্ড: `fmt -check` ও `validate` পাস; plan–এ অপ্রত্যাশিত destroy নেই; কোনো সিক্রেট বা `*.tfstate` Git–এ নেই; দ্বিতীয় plan–এ `No changes`। ইঙ্গিত: একটি রিসোর্স দিয়ে শুরু করে plan দেখুন, তারপর বাড়ান।

Assignment 7 • Volume 08

Assignment 7 — CI Pipeline: Plan on PR, Apply on Merge

৭.১ উদ্দেশ্য

CI পাইপলাইন — PR-এ plan, merge-এ apply: GitHub Actions: fmt, validate, plan, apply, approval gate সহ। সময়: ২–৪ ঘণ্টা। TechNova (`technova.example`) পরিবেশে কাজ করুন।

৭.২ ডেলিভারেবল ও গ্রহণযোগ্যতা

ডেলিভারেবল: Terraform কোড (`main.tf` , `variables.tf` , `outputs.tf` , `versions.tf` , `README.md`), এক পৃষ্ঠার পরিকল্পনা নোট, `plan` / `apply` প্রমাণ, ক্লাউড কাজে `destroy` প্রমাণ। মানদণ্ড: `fmt -check` ও `validate` পাস; plan-এ অপ্রত্যাশিত destroy নেই; কোনো সিক্রেট বা `*.tfstate` Git-এ নেই; দ্বিতীয় plan-এ `No changes` । ইঙ্গিত: একটি রিসোর্স দিয়ে শুরু করে plan দেখুন, তারপর বাড়ান।

Assignment 8 • Volume 08

Assignment 8 — Dynamic Security Group Module

৮.১ উদ্দেশ্য

ডাইনামিক সিকিউরিটি গ্রুপ মডিউল: `dynamic` ব্লক দিয়ে পোর্ট তালিকা থেকে নিয়ম, `for_each` ব্যবহার। সময়: ২–৪ ঘণ্টা। TechNova (`technova.example`) পরিবেশে কাজ করুন।

৮.২ ডেলিভারেবল ও গ্রহণযোগ্যতা

ডেলিভারেবল: Terraform কোড (`main.tf`, `variables.tf`, `outputs.tf`, `versions.tf`, `README.md`), এক পৃষ্ঠার পরিকল্পনা নোট, `plan` / `apply` প্রমাণ, ক্লাউড কাজে `destroy` প্রমাণ। মানদণ্ড: `fmt -check` ও `validate` পাস; plan–এ অপ্রত্যাশিত destroy নেই; কোনো সিক্রেট বা `*.tfstate` Git–এ নেই; দ্বিতীয় plan–এ `No changes`। ইঙ্গিত: একটি রিসোর্স দিয়ে শুরু করে plan দেখুন, তারপর বাড়ান।

Assignment 9 • Volume 08

Assignment 9 — Zero-Downtime Variable Change

৯.১ উদ্দেশ্য

জিরো-ডাউনটাইম ভ্যারিয়েবল পরিবর্তন: ইনস্ট্যান্স টাইপ বদলে plan পড়ে কী ধ্বংস হবে তা ব্যাখ্যা করা। সময়: ২–৪ ঘণ্টা। TechNova (`technova.example`) পরিবেশে কাজ করুন।

৯.২ ডেলিভারেবল ও গ্রহণযোগ্যতা

ডেলিভারেবল: Terraform কোড (`main.tf`, `variables.tf`, `outputs.tf`, `versions.tf`, `README.md`), এক পৃষ্ঠার পরিকল্পনা নোট, `plan` / `apply` প্রমাণ, ক্লাউড কাজে `destroy` প্রমাণ। মানদণ্ড: `fmt -check` ও `validate` পাস; plan–এ অপ্রত্যাশিত destroy নেই; কোনো সিক্রেট বা `*.tfstate` Git–এ নেই; দ্বিতীয় plan–এ `No changes`। ইঙ্গিত: একটি রিসোর্স দিয়ে শুরু করে plan দেখুন, তারপর বাড়ান।

Assignment 10 • Volume 08

Assignment 10 — Cost Estimation Report

১০.১ উদ্দেশ্য

খরচ অনুমান প্রতিবেদন: মাসিক খরচের টেবিল, সাশ্রয়ের উপায়, বাজেট অ্যালার্ট। সময়: ২–৪ ঘণ্টা। TechNova (`technova.example`) পরিবেশে কাজ করুন।

১০.২ ডেলিভারেবল ও গ্রহণযোগ্যতা

ডেলিভারেবল: Terraform কোড (`main.tf`, `variables.tf`, `outputs.tf`, `versions.tf`, `README.md`), এক পৃষ্ঠার পরিকল্পনা নোট, `plan` / `apply` প্রমাণ, ক্লাউড কাজে `destroy` প্রমাণ। মানদণ্ড: `fmt -check` ও `validate` পাস; plan–এ অপ্রত্যাশিত destroy নেই; কোনো সিক্রেট বা `*.tfstate` Git–এ নেই; দ্বিতীয় plan–এ `No changes`। ইঙ্গিত: একটি রিসোর্স দিয়ে শুরু করে plan দেখুন, তারপর বাড়ান।

Assignment 11 • Volume 08

Assignment 11 — Drift Detection Runbook

১১.১ উদ্দেশ্য

ড্রিফট শনাক্তকরণ রানবুক: হাতে রিসোর্স বদলে `plan` দিয়ে ড্রিফট ধরা ও ঠিক করা। সময়: ২–৪ ঘণ্টা। TechNova (`technova.example`) পরিবেশে কাজ করুন।

১১.২ ডেলিভারেবল ও গ্রহণযোগ্যতা

ডেলিভারেবল: Terraform কোড (`main.tf`, `variables.tf`, `outputs.tf`, `versions.tf`, `README.md`), এক পৃষ্ঠার পরিকল্পনা নোট, `plan` / `apply` প্রমাণ, ক্লাউড কাজে `destroy` প্রমাণ। মানদণ্ড: `fmt -check` ও `validate` পাস; plan–এ অপ্রত্যাশিত destroy নেই; কোনো সিক্রেট বা `*.tfstate` Git–এ নেই; দ্বিতীয় plan–এ `No changes`। ইঙ্গিত: একটি রিসোর্স দিয়ে শুরু করে plan দেখুন, তারপর বাড়ান।

Assignment 12 • Volume 08

Assignment 12 — Secure Variables & Secrets

১২.১ উদ্দেশ্য

নিরাপদ ভ্যারিয়েবল ও সিক্রেট: sensitive ভ্যারিয়েবল, `*.tfvars` gitignore, সিক্রেট ম্যানেজার। সময়: ২–৪ ঘণ্টা। TechNova (`technova.example`) পরিবেশে কাজ করুন।

১২.২ ডেলিভারেবল ও গ্রহণযোগ্যতা

ডেলিভারেবল: Terraform কোড (`main.tf`, `variables.tf`, `outputs.tf`, `versions.tf`, `README.md`), এক পৃষ্ঠার পরিকল্পনা নোট, `plan` / `apply` প্রমাণ, ক্লাউড কাজে `destroy` প্রমাণ। মানদণ্ড: `fmt -check` ও `validate` পাস; plan–এ অপ্রত্যাশিত destroy নেই; কোনো সিক্রেট বা `*.tfstate` Git–এ নেই; দ্বিতীয় plan–এ `No changes`। ইঙ্গিত: একটি রিসোর্স দিয়ে শুরু করে plan দেখুন, তারপর বাড়ান।

Assignment 13 • Volume 08

Assignment 13 — Module Registry-Style Versioning

১৩.১ উদ্দেশ্য

মডিউল ভার্সনিং: Git ট্যাগ দিয়ে সংস্করণ, `source = "git::...?ref=v1.0.0"`। সময়: ২–৪ ঘণ্টা। TechNova (`technova.example`) পরিবেশে কাজ করুন।

১৩.২ ডেলিভারেবল ও গ্রহণযোগ্যতা

ডেলিভারেবল: Terraform কোড (`main.tf`, `variables.tf`, `outputs.tf`, `versions.tf`, `README.md`), এক পৃষ্ঠার পরিকল্পনা নোট, `plan` / `apply` প্রমাণ, ক্লাউড কাজে `destroy` প্রমাণ। মানদণ্ড: `fmt -check` ও `validate` পাস; plan–এ অপ্রত্যাশিত destroy নেই; কোনো সিক্রেট বা `*.tfstate` Git–এ নেই; দ্বিতীয় plan–এ `No changes`। ইঙ্গিত: একটি রিসোর্স দিয়ে শুরু করে plan দেখুন, তারপর বাড়ান।

Assignment 14 • Volume 08

Assignment 14 — Disaster Recovery State Backup

১৪.১ উদ্দেশ্য

দুর্যোগ পুনরুদ্ধার: স্টেট ব্যাকআপ: স্টেট ব্যাকআপ, সংস্করণ, পুনরুদ্ধারের ধাপ ও পরীক্ষা। সময়: ২–৪ ঘণ্টা। TechNova (`technova.example`) পরিবেশে কাজ করুন।

১৪.২ ডেলিভারেবল ও গ্রহণযোগ্যতা

ডেলিভারেবল: Terraform কোড (`main.tf`, `variables.tf`, `outputs.tf`, `versions.tf`, `README.md`), এক পৃষ্ঠার পরিকল্পনা নোট, `plan` / `apply` প্রমাণ, ক্লাউড কাজে `destroy` প্রমাণ। মানদণ্ড: `fmt -check` ও `validate` পাস; plan–এ অপ্রত্যাশিত destroy নেই; কোনো সিক্রেট বা `*.tfstate` Git–এ নেই; দ্বিতীয় plan–এ `No changes`। ইঙ্গিত: একটি রিসোর্স দিয়ে শুরু করে plan দেখুন, তারপর বাড়ান।

Assignment 15 • Volume 08

Assignment 15 — Refactor Monolith to Modules

১৫.১ উদ্দেশ্য

মনোলিথ কোড মডিউলে ভাগ করা: বড় `main.tf`-কে network/compute/storage মডিউলে ভাগ, state mv ব্যবহার। সময়: ২–৪ ঘণ্টা। TechNova (`technova.example`) পরিবেশে কাজ করুন।

১৫.২ ডেলিভারেবল ও গ্রহণযোগ্যতা

ডেলিভারেবল: Terraform কোড (`main.tf`, `variables.tf`, `outputs.tf`, `versions.tf`, `README.md`), এক পৃষ্ঠার পরিকল্পনা নোট, `plan` / `apply` প্রমাণ, ক্লাউড কাজে `destroy` প্রমাণ। মানদণ্ড: `fmt -check` ও `validate` পাস; plan–এ অপ্রত্যাশিত destroy নেই; কোনো সিক্রেট বা `*.tfstate` Git–এ নেই; দ্বিতীয় plan–এ `No changes`। ইঙ্গিত: একটি রিসোর্স দিয়ে শুরু করে plan দেখুন, তারপর বাড়ান।

Project 1 • Volume 08

Project ১ — Modular Cloud Infrastructure

এই প্রজেক্টটি একটি বাস্তব, সম্পূর্ণ কাজ — ল্যাবের ছোট অনুশীলন নয়। পরিকল্পনা লিখুন, তারপর বাস্তবায়ন করুন।

১.১ উদ্দেশ্য

TechNova-র জন্য পুনর্ব্যবহারযোগ্য মডিউল দিয়ে সম্পূর্ণ ক্লাউড ইনফ্রাস্ট্রাকচার তৈরি করুন: নেটওয়ার্কিং, সাবনেট, সিকিউরিটি গ্রুপ / ফায়ারওয়াল, কম্পিউট, স্টোরেজ ও IAM। প্রতিটি মডিউল স্বাধীনভাবে পরীক্ষা ও সংস্করণ দেওয়া যাবে।

১.২ ফাইল কাঠামো ও মূল কোড

```
technova-infra/
├── modules/network, compute, storage, iam/
├── environments/dev, prod/
├── versions.tf, README.md, .gitignore
```

```
# modules/network/main.tf
variable "cidr_block" { type = string }
variable "public_subnet_cidrs" { type = list(string) }
resource "aws_vpc" "main" {
  cidr_block = var.cidr_block
  tags = { Name = "technova-vpc" }
}
resource "aws_subnet" "public" {
  count                   = length(var.public_subnet_cidrs)
  vpc_id                  = aws_vpc.main.id
  cidr_block              = var.public_subnet_cidrs[count.index]
  map_public_ip_on_launch = true
}
output "vpc_id" { value = aws_vpc.main.id }
output "public_subnet_ids" { value = aws_subnet.public[*].id }
```

```
# Root composition
module "network" {
  source = "./modules/network"
  cidr_block = "10.0.0.0/16"
  public_subnet_cidrs = ["10.0.1.0/24", "10.0.2.0/24"]
}
module "compute" {
  source = "./modules/compute"
  vpc_id = module.network.vpc_id
  subnet_ids = module.network.public_subnet_ids
}
```

১.৩ ইনপুট, আউটপুট, নির্ভরতা ও ভার্সনিং

ইনপুটে `description`, `type` ও `validation`; আউটপুটে শুধু প্রয়োজনীয় মান। নির্ভরতা আউটপুটের মাধ্যমে প্রকাশ করুন; `depends_on` শেষ উপায়। Git ট্যাগ দিয়ে সংস্করণ দিন (`ref=v1.0.0`)।

> **WARNING:** ক্লাউড খরচ হবে — শেষে `terraform destroy` চালান।

ডেলিভারেবল: সম্পূর্ণ রিপোজিটরি, প্রতিটি মডিউলের README, plan/apply/destroy প্রমাণ। `fmt` ও `validate` পাস এবং দ্বিতীয় plan–এ No changes হলে গ্রহণযোগ্য।

Project 2 • Volume 08

Project ২ — Multi-Environment Infrastructure

এই প্রজেক্টটি একটি বাস্তব, সম্পূর্ণ কাজ — ল্যাবের ছোট অনুশীলন নয়। পরিকল্পনা লিখুন, তারপর বাস্তবায়ন করুন।

২.১ উদ্দেশ্য

Development, Staging ও Production — তিন পরিবেশে একই মডিউল পুনর্ব্যবহার করে আলাদা ইনফ্রাস্ট্রাকচার তৈরি করুন। ভ্যারিয়েবল, স্টেট, অ্যাক্সেস ও নিরাপত্তা আলাদা হবে, কোড একটাই থাকবে।

২.২ পরিবেশ-ভিত্তিক ভ্যারিয়েবল ও স্টেট

```
# environments/dev/terraform.tfvars
environment = "dev"
instance_type = "t3.micro"
# environments/prod/terraform.tfvars
environment = "prod"
instance_type = "t3.small"
```

আলাদা স্টেট কী ব্যবহার করুন: `dev/terraform.tfstate`, `prod/terraform.tfstate`। এক স্টেটে দুই পরিবেশ মেশাবেন না। রিমোট ব্যাকএন্ডে এনক্রিপশন ও লকিং চালু করুন; prod-এ apply শুধু নির্দিষ্ট ভূমিকার, পাবলিক অ্যাক্সেস বন্ধ, `Environment=prod` ট্যাগ বাধ্যতামূলক।

২.৩ Workspaces বনাম আলাদা Root কনফিগারেশন

পদ্ধতি	সুবিধা	অসুবিধা	কখন
Workspaces	একই কোড, দ্রুত বদল	ভুল workspace-এ apply-এর ঝুঁকি	ছোট, একই রকম পরিবেশ
আলাদা root কনফিগারেশন	স্পষ্ট পৃথকীকরণ, আলাদা ব্যাকএন্ড ও অনুমতি	কিছু পুনরাবৃত্তি	prod-এর মতো গুরুত্বপূর্ণ পরিবেশ

এই প্রজেক্টে আলাদা root কনফিগারেশন ব্যবহার করুন; ট্রেড-অফ নকশা নোটে লিখুন।

ডেলিভারেবল: তিন পরিবেশের কনফিগারেশন, ব্যাকএন্ড কনফিগ, ডেপ্লয়মেন্ট রানবুক, ট্রেড-অফ নোট, plan/apply/destroy প্রমাণ।

> **WARNING:** তিন পরিবেশ মানে তিনগুণ খরচের সম্ভাবনা — পরীক্ষার পর destroy করুন।

Project 3 • Volume 08

Project ৩ — TechNova Infrastructure Automation Platform

এই প্রজেক্টটি একটি বাস্তব, সম্পূর্ণ কাজ — ল্যাবের ছোট অনুশীলন নয়। পরিকল্পনা লিখুন, তারপর বাস্তবায়ন করুন।

৩.১ উদ্দেশ্য

এন্টারপ্রাইজ-ধাঁচের IaC ওয়ার্কফ্লো: রিপোজিটরি, মডিউল, রিমোট স্টেট, লকিং, প্রোভাইডার সংস্করণ, ভ্যালিডেশন, নিরাপত্তা স্ক্যানিং, CI/CD, plan পর্যালোচনা, approval gate, প্রোডাকশন apply, ডকুমেন্টেশন ও খরচ ব্যবস্থাপনা।

৩.২ কাঠামো, স্টেট ও CI/CD

```
technova-platform/
├── modules/network, compute, storage, iam/
├── environments/dev, staging, prod/
├── policies/  # checkov / OPA
├── .github/workflows/terraform.yml
├── backend.hcl, README.md
# backend.hcl
bucket = "CHANGE-ME-technova-tfstate"
key = "prod/terraform.tfstate"
region = "ap-south-1"
dynamodb_table = "CHANGE-ME-technova-locks"
encrypt = true
```

```
# .github/workflows/terraform.yml (সংক্ষেপ)
jobs:
  terraform:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
        with: { terraform_version: "1.9.8" }
      - run: terraform fmt -check -recursive
      - run: terraform init -backend=false
      - run: terraform validate
      - run: terraform plan -no-color
```

PR–এ `fmt` → `validate` → `tflint` → `checkov` → `plan`; plan PR–এ মন্তব্য হিসেবে যাবে। merge–এ staging apply, prod–এ মানুষের অনুমোদন ছাড়া apply নয়। উচ্চ-ঝুঁকির স্ক্যান finding থাকলে merge আটকে যাবে।

ডেলিভারেবল: সম্পূর্ণ রিপোজিটরি, CI ওয়ার্কফ্লো, নীতি ফাইল, রানবুক, খরচ প্রতিবেদন। PR–এ plan দেখা গেলে, prod apply–এ approval লাগলে এবং স্ক্যান ছাড়া merge না হলে গ্রহণযোগ্য।

Capstone • Volume 08

Capstone — TechNova Multi-Environment Infrastructure Platform

TechNova (`technova.example`)-এর জন্য একটি সম্পূর্ণ ইনফ্রাস্ট্রাকচার অটোমেশন ব্যবস্থা ডিজাইন ও বাস্তবায়ন করুন — একটি বাস্তব অ্যাপ্লিকেশন পরিবেশ (ওয়েব অ্যাপ, API, ডেটাবেস, স্ট্যাটিক সম্পদ) Development, Staging ও Production-এ প্রভিশন ও পরিচালনা করবে।

১.১ আর্কিটেকচার

```
Developer --PR--> GitHub --plan--> Reviewer --approve--> Apply
                Modules (network/compute/storage/iam)
                  Remote State (S3 + DynamoDB lock)
                     Cloud: dev | staging | prod
```

১.২ যা যা করতে হবে

পরিকল্পনা: ক্লাউড প্রোভাইডার নির্বাচন ও কারণ, নেটওয়ার্ক পরিকল্পনা (CIDR, সাবনেট)। **মডিউল:** networking, compute, storage, IAM — পুনর্ব্যবহারযোগ্য, সংস্করণসহ। **পরিবেশ:** আলাদা ভ্যারিয়েবল ও স্টেট। **নিরাপত্তা:** ভ্যালিডেশন, `checkov` / `tfsec`, সীমিত IAM। **CI/CD:** PR-এ plan, merge-এ staging apply, prod-এ approval gate। **ড্রিফট:** নিয়মিত `plan` দিয়ে হাতে বদল ধরা ও ঠিক করার রানবুক। **খরচ:** বাজেট, অ্যালার্ট, মাসিক প্রতিবেদন। **ডকুমেন্টেশন ও পরিষ্কার:** রানবুক ও `destroy` প্রমাণ।

> **WARNING:** ক্লাউড খরচ হবে — বাজেট অ্যালার্ট সেট করুন, ছোট সাইজ ব্যবহার করুন, শেষে সব পরিবেশ destroy করুন।

১.৩ গ্রহণযোগ্যতা ও গ্রেডিং রুব্রিক

তিন পরিবেশের plan সফল; একই মডিউল তিন পরিবেশে চলে; স্টেট রিমোট ও লক করা; CI-এ fmt, validate, স্ক্যান ও plan; prod apply-এ approval; ড্রিফট প্রমাণ; কোনো সিক্রেট Git-এ নেই; destroy প্রমাণ আছে।

মানদণ্ড	ওজন	চমৎকার (৯০–১০০%)	ভালো (৭০–৮৯%)	উন্নতি প্রয়োজন (<৭০%)
আর্কিটেকচার ও মডিউল	২৫%	পরিষ্কার, পুনর্ব্যবহারযোগ্য, সংস্করণসহ	মডিউল আছে, কিছু পুনরাবৃত্তি	মনোলিথ, পুনর্ব্যবহার নেই
পরিবেশ ও স্টেট	২০%	আলাদা স্টেট, লকিং, এনক্রিপশন	আলাদা স্টেট, লকিং নেই	একই বা লোকাল স্টেট
নিরাপত্তা ও ভ্যালিডেশন	২০%	স্ক্যান পাস, সীমিত IAM	স্ক্যান চলে, কিছু finding	

				স্ক্যান নেই বা খোলা অ্যাক্সেস
CI/CD ও অনুমোদন	২০%	PR plan, approval gate, সম্পূর্ণ প্রমাণ	CI আছে, approval নেই	CI নেই
ডকুমেন্টেশন, খরচ ও পরিষ্কার	১৫%	সম্পূর্ণ রানবুক, খরচ প্রতিবেদন, destroy প্রমাণ	আংশিক ডকুমেন্টেশন	ডকুমেন্টেশন বা destroy প্রমাণ নেই

মোট ১০০-তে ৭০ বা বেশি পেলে উত্তীর্ণ। নিজে মূল্যায়ন করুন, তারপর সহশিক্ষার্থীকে দিয়ে পর্যালোচনা করান।

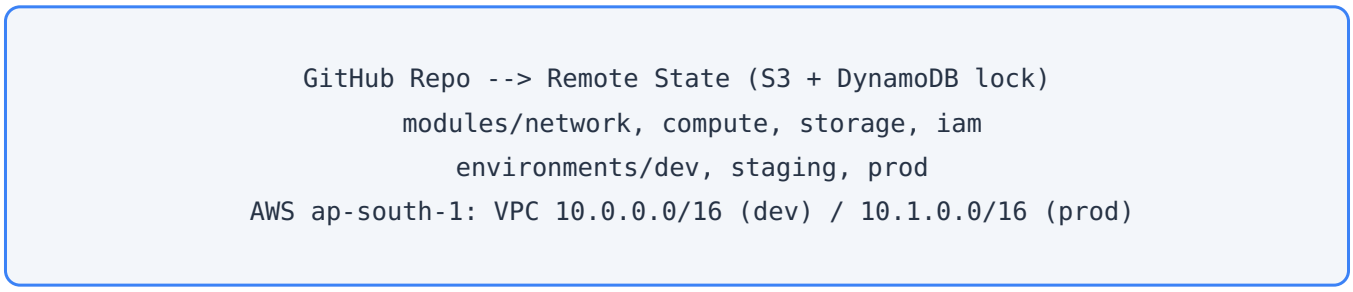
Solution Guide • Volume 08

Solution Guide — Capstone Reference Solution

> **WARNING:** এটি রেফারেন্স সমাধান — ক্যাপস্টোন নিজে চেষ্টা করার পরে পড়ুন। আগে পড়লে শেখা হবে না।

AWS-ভিত্তিক মাল্টি-এনভায়রনমেন্ট প্ল্যাটফর্মের রেফারেন্স। প্রোভাইডার ভিন্ন হলেও কাঠামো ও ওয়ার্কফ্লো একই। সব মান কাল্পনিক (`CHANGE-ME-...`, `technova.example`, `203.0.113.x`)।

১.১ আর্কিটেকচার ও রিপোজিটরি




```
technova-platform/
├── modules/network, compute, storage, iam/
├── environments/dev, staging, prod/
├── policies/, .github/workflows/terraform.yml
├── backend.hcl, README.md
```

১.২ মূল মডিউল কোড

```
# modules/network/main.tf
variable "cidr_block" { type = string }
variable "public_subnet_cidrs" { type = list(string) }
resource "aws_vpc" "main" {
  cidr_block = var.cidr_block
  enable_dns_hostnames = true
  tags = { Name = "technova-vpc" }
}
resource "aws_subnet" "public" {
  count = length(var.public_subnet_cidrs)
  vpc_id = aws_vpc.main.id
  cidr_block = var.public_subnet_cidrs[count.index]
  map_public_ip_on_launch = true
}
output "vpc_id" { value = aws_vpc.main.id }
output "public_subnet_ids" { value = aws_subnet.public[*].id }
```

```
# environments/prod/main.tf
module "network" {
  source = "../../modules/network"
  cidr_block = "10.1.0.0/16"
  public_subnet_cidrs = ["10.1.1.0/24", "10.1.2.0/24"]
```

```
}
module "compute" {
  source = "../../modules/compute"
  vpc_id = module.network.vpc_id
  subnet_ids = module.network.public_subnet_ids
}
```

১.৩ ব্যাকএন্ড, CI ও রানবুক

```
# backend.hcl
bucket = "CHANGE-ME-technova-tfstate"
key = "prod/terraform.tfstate"
region = "ap-south-1"
dynamodb_table = "CHANGE-ME-technova-locks"
encrypt = true
```

```
# .github/workflows/terraform.yml (সংক্ষেপ)
jobs:
  terraform:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
        with: { terraform_version: "1.9.8" }
      - run: terraform fmt -check -recursive
      - run: terraform init -backend=false
      - run: terraform validate
      - run: checkov -d . --framework terraform
      - run: terraform plan -no-color
```

রানবুক: ডেপ্লয় — শাখা → PR → plan পর্যালোচনা → merge → staging apply → prod approval → prod apply। ড্রিফট — সাপ্তাহিক `plan`; অপ্রত্যাশিত পরিবর্তন কোডে আনুন বা apply করে ঠিক করুন। জরুরি ফিরে যাওয়া — আগের কমিটে ফিরে plan ও apply। পরিষ্কার — `terraform destroy -auto-approve` ও বিলিং যাচাই।

> **শেষ কথা:** নিজের সমাধানের সঙ্গে মিলিয়ে দেখুন — কোথায় আপনার নকশা ভালো, কোথায় ভিন্ন, আর কেন। ভিন্ন হওয়া মানেই ভুল নয়; কারণ লিখতে পারলেই শিখেছেন।